The University of Oklahoma
School of Computer Science
CS 3113-001 - Intro to Operating Systems - Spring 2026
Syllabus

Instructor: Dr. Egawati Panjei, egawati.panjei@ou.edu

Instructor Office Location: Devon Energy Hall, DEH 235
Instructor Office Hours:
- Tuesdays/Thursdays: 3 - 4 PM
- Wednesdays: 11 AM - 12 PM

Class Location: Felgar Hall 0300
Class Time: 1:30 to 2:45 PM (Tuesdays and Thursdays)

Course Prerequisite: C S 2413 or C S 2414, and C S 2813 or MATH 2513, and C S 2614 or ECE 3223.

Textbook: Abraham Silberschatz, Peter Baer Galvin, Greg Gagne: Operating System Concepts, 10th Edition. Wiley 2018, ISBN 978-1-118-06333-0

Course Contents

Topics	Chapters
OS Overview	1, 2
Process Management	3, 4, 5
Process Synchronization	6, 7, 8
Memory Management	9, 10
Storage Management	11, 12
File Systems	13, 14, 15
Security and Protection	16

Learning Management System/Website: https://canvas.ou.edu/courses/489660

Course Catalog Description: An introduction to the major concepts and techniques of designing and implementing operating systems, including memory management, process management, information management, and computer security. Class projects require the design and implementation of software systems. A UNIX family operating system will be used.

Course Format: This course will be in a traditional in-class format.

Course Activities, Assignments, and Assigning Grades: Students are required to actively participate during classes and take weekly quizzes, two exams and a final, and programming projects. The course letter grade will be assigned based on the overall percentage: >= 90 (A), >=80 and < 90 (B), >=70 and < 80 (C),

>=60 and < 70 (D), and <60 (F). The instructor reserves the right to lower this grade scale if deemed appropriate. The allocation of percentages is given below:

	Percentages
In Class Participation	10 %
Weekly Quizzes	16%
Exam 1	18%
Exam 2	18%
Final	23%
Programming Projects	15%

Bonus Points: The instructor may offer opportunities for students to earn bonus points, up to a maximum of **5%**.

ABET Outcomes of Instruction in C S 3113:

> Design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of the program's discipline.

Learning Outcomes: Upon successful completion of this course, students will be able to:

1. Core Operating System Concepts
    - Explain the fundamental principles of operating systems, including process management, memory management, file systems, and security.
2. Process and Thread Management
    - Describe and implement concepts related to processes, threads, and concurrency, including context switching, scheduling algorithms, and synchronization mechanisms.
3. CPU Scheduling and Synchronization
    - Analyze different CPU scheduling algorithms and evaluate their efficiency under various workload conditions.
    - Apply synchronization techniques such as locks, semaphores, and monitors to solve concurrency problems.
4. Memory Management and Virtual Memory
    - Explain memory allocation strategies, paging, segmentation, and virtual memory concepts.
5. File Systems and Storage Management
    - Describe and compare different file system architectures and access methods.
    - Implement file I/O operations and understand disk scheduling techniques.
6. Security and Protection Mechanisms
    - Understand fundamental security concerns in operating systems, including access control, authentication, and system protection techniques.
7. Programming for OS Concepts in C/C++
    - Write C/C++ programs to implement and experiment with various operating system components, including process creation, inter-process communication (IPC), threading, memory management, and file system operations.

8. Performance Analysis and Optimization
    - Apply principles of performance evaluation to analyze OS algorithms and optimize system performance.
9. Critical Thinking and Problem-Solving Skills
    - Identify and solve real-world operating system-related challenges through programming assignments (projects).

Course Goals: The goals of this course are to:

1. Provide a Strong Theoretical Foundation in Operating Systems
    - Introduce students to the core principles, design, and implementation of modern operating systems.
    - Cover key concepts such as process management, memory management, file systems, security, and system performance.
2. Enhance Understanding of Process and Resource Management
    - Teach students how operating systems handle multiple processes, scheduling, synchronization, and inter-process communication.
    - Provide insights into concurrency challenges and solutions.
3. Build Competency in Memory and Storage Management
    - Help students understand how operating systems allocate, manage, and protect memory using techniques like paging and segmentation.
    - Explore file system structures, disk management, and I/O handling.
4. Expose Students to OS Security and Protection Mechanisms
    - Introduce key security principles, including access control, authentication, and protection mechanisms within an OS.
    - Discuss real-world threats and mitigation strategies.
5. Develop Practical Programming Skills for OS Concepts
    - Enable students to write and analyze C/C++ programs that interact with various OS components.
    - Reinforce theoretical concepts through hands-on programming assignments.
6. Develop Critical Thinking for System Design and Optimization
    - Encourage students to evaluate trade-offs in OS design choices and analyze system performance.
    - Foster problem-solving skills for optimizing operating system performance.
7. Prepare Students for Advanced Courses and Industry Applications
    - Equip students with the knowledge needed for advanced coursework in systems, networking, security, and distributed computing.
    - Provide skills relevant to real-world software development, system administration, and research in computing.

Tentative Schedule

*** Exams 1 and 2, Final Exam, and Quizzes schedules are fixed

Week	Day	Date	Topics	Quiz	Notes
1	Tuesday	25-Aug-26	Syllabus, OS Overview (Chapter 1)		
	Thursday	27-Aug-26	OS Overview (Chapter 1 and 2)		
2	Tuesday	1-Sep-26	OS Overview (Chapter 2)		**Project 0 Assigned**
	Thursday	3-Sep-26	Processes (Chapter 3)	Quiz 1	
3	Tuesday	8-Sep-26	Processes (Chapter 3)		
	Thursday	10-Sep-26	Processes (Chapter 3)	Quiz 2	
4	Tuesday	15-Sep-26	CPU Scheduling (Chapter 5)		**Project 0 Due, Project 1 Assigned**
	Thursday	17-Sep-26	No Lecture		Please go to engineering career fair!
5	Tuesday	22-Sep-26	CPU Scheduling (Chapter 5)		
	Thursday	24-Sep-26	CPU Scheduling (Chapter 5)	Quiz 3	
6	Tuesday	29-Sep-26	Main Memory (Chapter 9)		
	Thursday	1-Oct-26	**Exam 1**		Exam 1 is held during class time
7	Tuesday	6-Oct-26	Main Memory (Chapter 9)		**Project 1 Due, Project 2 Assigned**
	Thursday	8-Oct-26	Main Memory (Chapter 9)	Quiz 4	
8	Tuesday	13-Oct-26	Virtual Memory (Chapter 10)		
	Thursday	15-Oct-26	Virtual Memory (Chapter 10)	Quiz 5	
9	Tuesday	20-Oct-26	Virtual Memory (Chapter 10)		
	Thursday	22-Oct-26	Threads and Concurrency (Chapter 4)	Quiz 6	
10	Tuesday	27-Oct-26	Threads and Concurrency (Chapter 4)		**Project 2 Due, Project 3 Assigned**
	Thursday	29-Oct-26	No Lecture	Quiz 7 Online	Norman Campus classes are canceled due to the Showtime on the Soval concert.
11	Tuesday	3-Nov-26	Synchronization (Chapter 6, 7)		
	Thursday	5-Nov-26	Synchronization (Chapter 6, 7)	Quiz 8	
12	Tuesday	10-Nov-26	Deadlocks (Chapter 8)		

	Thursday	12-Nov-26	**Exam 2**		Exam 2 is held during class time
13	Tuesday	17-Nov-26	Deadlocks (Chapter 8)		
	Thursday	19-Nov-26	Storage Structure (Chapter 11 and 12)	Quiz 9	
14	Tuesday	24-Nov-26	Storage Structure (Chapter 11 and 12)		**Project 3 Due**
	Thursday	26-Nov-26	Thanksgiving No Class		
15	Tuesday	1-Dec-26	File Systems (Chapter 13,14,15)		
	Thursday	3-Dec-26	File Systems (Chapter 13,14,15)	Quiz 10	
16	Tuesday	8-Dec-26	Security and Protection (Chapter 16, 17)		
	Thursday	10-Dec-26	Final Exam Review		
17	Tuesday	15-Dec-26	**Final Exam**		**Felgar Hall 0300 1:30 - 3:30 PM**

You've made it to page 5! Keep reading — you may find a hidden gem worth your first bonus point.

Course Expectations:

Come ready to participate: attend lectures, join in discussions, and ask questions when something's unclear. Programming assignments should be completed independently and require a solid foundation in C/C++. Please follow the coding standards, document your work clearly, meet the project specifications, and avoid sharing code — academic integrity matters here.

Doing well in this course comes down to consistency: keep up with the readings, review your lecture notes, and practice problem-solving regularly rather than cramming. If a topic isn't clicking, reach out for help sooner rather than later. Good time management goes a long way — start early, build in time for debugging, and avoid last-minute work whenever possible.

Discussing ideas and problem-solving strategies with classmates is encouraged, but the work you submit should be your own. Be sure to follow all course policies, and don't hesitate to reach out to the instructor or TAs for support — taking ownership of your learning is part of the process. The critical thinking and problem-solving skills you build here will serve you well beyond this course, both academically and in industry.

Course Reflection Survey: You'll receive a Course Reflection Survey at the end of each semester for each course that you are enrolled in. I strongly encourage you to complete this survey. Your feedback can help me adjust my class for future semesters to help other students be successful. Your feedback is confidential, and I will only receive it after final grades are due. Course Reflection Survey results may also factor into teaching evaluations and annual performance reviews and are shared with department and program chairs.

Copyright Statement: Sessions of this course may be recorded or live-streamed. These recordings are the intellectual property of the individual faculty member and may not be shared or reproduced without the explicit, written consent of the faculty member. In addition, privacy rights of others such as students, guest lecturers, and providers of copyrighted material displayed in the recording may be of concern. Students may not share any course recordings with individuals not enrolled in the class or upload them to any other online environment

Sometime between the first week of classes and the Wednesday before Thanksgiving, send the instructor an email (not a Canvas message) with the exact subject line: *CS 3113-001 OS is awesome.* Keep reading the syllabus/handout — further down you will find the specific content that must be included in the body of this email.

Course Policies

Quizzes: There will be a quiz every Thursday, starting in Week 2, except during weeks when an exam is scheduled. The Quiz will cover topics discussed on previous Thursday and Tuesday lectures. All quizzes are conducted on paper and are closed book and closed electronics except when we need to do a Take Home Quiz. You may use **one two-sided, one-page letter-size sheet** of notes. Quiz grades will be posted on Grade Scope and transferred to Canvas after Exam 1, Exam 2, and during the last week of classes. To support your overall success, **your two lowest quiz scores will be dropped** at the end of the semester**.** Makeup quizzes will not be offered; the two dropped quizzes are intended to accommodate any missed quizzes for any reason.

Exams: All exams will be conducted on paper and taken in class. Exams are closed book and closed electronics (no laptops or electronic devices). Only handwritten notes are allowed.
For each exam, you may bring a number of handwritten note pages equal to:

- **(Number of quizzes completed before the exam) + 1**

For the final exam, you may bring a maximum number of handwritten note pages equal to:

- **(Total number of quizzes) + 3**

The final exam will be comprehensive and will cover all topics discussed during the semester. Exams 1 and 2 will each cover only the topics discussed prior to the exam. Topics covered in an earlier exam will not be included in later exams (for example, topics from Exam 1 will not appear on Exam 2).

There is no makeup for the final exam. However, if you miss Exam 1 or 2, please refer to the following rules.

Makeup Exam – Category 1 (Planned Absences)
If you must miss Exam 1 or 2 due to a university approved events (e.g. attending conference, job interview, religious observance), you must email the instructor at least one week before the exam with the **subject line**: *C S 3313: Missing Exam X* (replace *X* with the exam number).

Your email must include documentation explaining why you will miss the exam. With **prior approval from the instructor**, you may take a makeup exam within one week after the scheduled exam. The instructor will arrange the date and time for the makeup exam.

Makeup Exam – Category 2 (Emergencies)
If you miss Exam 1 or 2 due to an emergency and **receive approval from the instructor**, you may take the makeup exam together with Category 1 students, if possible. If you are unable to take the Category 1 makeup exam, the percentage weight of the missed exam will be added to your final exam grade. In this case, no additional makeup exam will be given.

Programming Projects:
Projects 1-3 must be coded in C or C++. We will use ANSI C or C++, so if your program compiles with any G++ or GCC compiler, you are set to go.

1. You will use GradeScope to submit the source program.
2. For every 24 hours late, you will be deducted 5% of the grade of the programming project. Any project that is more than 5 days late will not be evaluated.
3. You are better off submitting a working project on the fifth day rather than one that does not work on the due date.
4. Programs have to be documented clearly. Programs that lack or are weak in documentation will receive a deduction of up to 30% of the grade. Follow the documentation methods used in programs in your data structures book.
5. You will demo your project to the grader during the grader-assigned special office hours **if the grader so wishes**. Graders are not responsible for debugging your programs.
6. The project specifications presented by the instructor may not contain all the implementation details. It is your responsibility to understand the specifications thoroughly. Please ensure that all relevant questions regarding the project are asked during class time.
7. **All coding assignments must be completed independently**. You may discuss ideas with others or consult external resources (such as the internet or generative AI tools); however, you must write all code yourself. Do not copy or share code with anyone. All discussions and external influences must be clearly documented in your submission.
8. In addition to the above general evaluation policies, each programming project will have a set of specifications that must be met.

In Class Participation: Regular attendance is essential for success in this course, as active participation in lectures and discussions is key to understanding operating system concepts. In class participation will be recorded in each class using **Top Hat** and will count for **10%** of the overall course grade. Recording attendance through Top Hat while not physically present in class is considered cheating and will be handled according to the academic misconduct policy. To accommodate absences for any reason, students need only earn 85% of the total possible in-class participation points to receive full credit (the full 10%) for this component.

Students are responsible for ensuring their participation is recorded correctly. Absences due to emergencies or university-approved events should be communicated in advance whenever possible and supporting documentation may be required for consideration.

While lecture recordings may be available for some sessions, they are not a substitute for in-class participation. Students are encouraged to engage actively in discussions and ask questions, as interaction is a key component of learning in this course.

Incompletes: The grade of "I" is intended for the rare circumstance when a student who has been successful in a class has an unexpected event shortly before the class's end. We will not consider giving a student a grade of "I" unless the following three conditions have been met:

1. It is within two weeks of the end of the semester.
2. The student has a grade of C or better in the class.
3. The reason that the student cannot complete the class is properly documented and compelling.

Classroom Conduct: Because cell phones and laptops can distract substantially from the classroom experience, students are asked not to use either during class except when required as part of a classroom exercise. Disruptions of the class will also not be permitted. In the case of disruptive behavior, we may ask that you leave the classroom and charge you with violating the Student Code of Responsibilities and Conduct. Examples of disruptive behavior include:

- Allowing a cell phone or pager to repeatedly beep audibly.
- Playing music or computer games during class in such a way that they are visible or audible to other class members.
- Exhibiting erratic or irrational behavior.
- Behavior that distracts the class from the subject matter or discussion.
- Making physical or verbal threats to a faculty member, teaching assistant, or class member.

Proper Academic Conduct: Feel free to discuss all assignments with the instructors or the TAs. Code (projects and assignment): you may discuss solutions with other students. However:

1. You may not look at or share code with others.
2. If you discuss a solution with anyone, you must document their names in your assignment.
3. You must document this in your code if you use external resources (e.g., Piazza, StackOverflow.com, ChatGPT, or other LLMs).

Make sure that your computer account is properly protected. Use an appropriate password, and do not give your friends access to your account or computer system. Do not leave printouts, computers, or thumb drives around a laboratory where others might access them. Programming projects will be checked by software designed to detect collaboration.

Upon the first documented occurrence of inappropriate collaborative work or of taking a solution from a network resource, the instructors will report the academic misconduct to the Campus Judicial Coordinator. The procedure is documented in the University of Oklahoma Academic Misconduct Code (http://integrity.ou.edu). The provider and receiver of a solution will be treated equally in the misconduct process.

Generative AI Policy: Students are permitted to use Generative AI tools, such as ChatGPT and Copilot, as aids in understanding operating system concepts, debugging code, and generating ideas for programming assignments. However, all work submitted must reflect the student's own understanding and effort. If Generative AI is used to assist in coding, problem-solving, or explanation, students must explicitly acknowledge its use by adding comments in their code or submissions detailing what aspects were AI-assisted. Directly copying AI-generated solutions without comprehension or modification is not acceptable and may be considered academic misconduct. This course aims to develop independent problem-solving skills, so while AI can be a helpful tool, it should complement learning rather than replace critical thinking

and hands-on programming experience. Students should consult the instructor if they have questions about appropriate AI usage.
Within that message (sent between the first week of classes and the Wednesday before Thanksgiving), include a brief story explaining why you chose your current major. If you have any pets or favorite animals, also attach a picture of them.

University Policies

Mental Health Support Services

Support is available for any student experiencing mental health issues that are impacting their academic success. Students can either been seen at the University Counseling Center (UCC) located on the second floor of Goddard Health Center or receive 24/7/365 crisis support from a licensed mental health provider through TimelyCare. To schedule an appointment or receive more information about mental health resources at OU please call the UCC at 405-325-2911 or visit University Counseling Center. The UCC is located at 620 Elm Ave., Room 201, Norman, OK 73019.

Title IX Resources and Reporting Requirement

The University of Oklahoma faculty are committed to creating a safe learning environment for all members of our community, free from sex-based discrimination, including sexual harassment, domestic and dating violence, sexual assault, and stalking, in accordance with Title IX. There are resources available to those impacted, including: speaking with someone confidentially about your options, medical attention, counseling, reporting, academic support, and safety plans. If you have (or someone you know has) experienced any form of sex-based discrimination or violence and wish to speak with someone confidentially, please contact OU Advocates (available 24/7 at 405-615-0013) or University Counseling Center (M-F 8 a.m. to 5 p.m. at 405-325-2911).

Because the University of Oklahoma is committed to the safety of you and other students, and because of our Title IX obligations, I, as well as other faculty, Graduate Assistants, and Teaching Assistants, are mandatory reporters. This means that we are obligated to report sex-based violence that has been disclosed to us to the Institutional Equity Office. This means that we are obligated to report sex-based violence that has been disclosed to us to the Institutional Equity Office. This includes disclosures that occur in: class discussion, writing assignments, discussion boards, emails and during Student/Office Hours. You may also choose to report directly to the Institutional Equity Office. After a report is filed, the Title IX Coordinator will reach out to provide resources, support, and information and the reported information will remain private. For more information regarding the University's Title IX Grievance procedures, reporting, or support measures, please visit Institutional Equity Office at 405-325-3546.

Adjustments for Pregnancy and Related Issues

Should you need modifications or adjustments to your course requirements because of pregnancy or a pregnancy-related condition, please request modifications via the Institutional Equity Office website or call the Institutional Equity Office at 405/325-3546 as soon as possible. Also, see the Institutional Equity Office FAQ on Pregnant and Parenting Students' Rights for answers to commonly asked questions.

Reasonable Accommodation Policy

The University of Oklahoma (OU) is committed to the goal of achieving equal educational opportunity and full educational participation for students with disabilities. If you have already established reasonable accommodations with the Accessibility and Disability Resource Center (ADRC), please log into AIM to request your semester accommodations as soon as possible and contact me privately, so that we have adequate time to arrange your approved academic accommodations.

If you have not yet established services through ADRC, but have a documented disability and require accommodations, please complete ADRC's pre-registration form to begin the registration process. ADRC facilitates the interactive process that establishes reasonable accommodations for students at OU. For more information on ADRC registration procedures, please review their website. You may also contact them at (405)325-3852 or adrc@ou.edu, or visit www.ou.edu/adrc for more information.
Note: disabilities may include, but are not limited to, mental health, chronic health, physical, vision, hearing, learning and attention disabilities, pregnancy-related. ADRC can also support students experiencing temporary medical conditions.

Religious Observance

It is the policy of the University to excuse the absences of students that result from religious observances and to reschedule examinations and additional required classwork that may fall on religious holidays, without penalty. [See Faculty Handbook 3.15.2]

Final Exam Preparation Period

Pre-finals week will be defined as the seven calendar days before the first day of finals. Faculty may cover new course material throughout this week. For specific provisions of the policy please refer to OU's Final Exam Preparation Period policy.

Emergency Protocol

During an emergency, there are official university procedures that will maximize your safety.

Severe Weather

If you receive an OU Alert to seek refuge or hear a tornado siren that signals severe weather.

1. Look for severe weather refuge location maps located inside most OU buildings near the entrances.
2. Seek refuge inside a building. Do not leave one building to seek shelter in another building that you deem safer. If outside, get into the nearest building.
3. Go to the building's severe weather refuge location. If you do not know where that is, go to the lowest level possible and seek refuge in an innermost room. Avoid outside doors and windows.
4. Get in, Get Down, Cover Up
5. Wait for official notice to resume normal activities.

Additional Weather Safety Information is available through the Department of Campus Safety.

The University of Oklahoma Active Threat Guidance

The University of Oklahoma embraces a Run, Hide, Fight strategy for active threats on campus. This strategy is well known, widely accepted, and proven to save lives. To receive emergency campus alerts, be sure to update your contact information and preferences in the account settings section at one.ou.edu.

RUN: Running away from the threat is usually the best option. If it is safe to run, run as far away from the threat as possible. Call 911 when you are in a safe location and let them know from which OU campus you're calling from and location of active threat.

HIDE: If running is not practical, the next best option is to hide. Lock and barricade all doors; turn of all lights; turn down your phone's volume; search for improvised weapons; hide behind solid objects and walls; and hide yourself completely and stay quiet. Remain in place until law enforcement arrives. Be patient and remain hidden.

FIGHT: If you are unable to run or hide, the last best option is to fight. Have one or more improvised weapons with you and be prepared to attack. Attack them when they are least expecting it and hit them where it hurts most: the face (specifically eyes, nose, and ears), the throat, the diaphragm (solar plexus), and the groin.

Please save OUPD's contact information in your phone.

NORMAN campus: *For non-emergencies call (405) 325-1717. For emergencies call (405) 325-1911 or dial 911.*

TULSA campus: *For non-emergencies call (918) 660-3900. For emergencies call (918) 660-3333 or dial 911.*

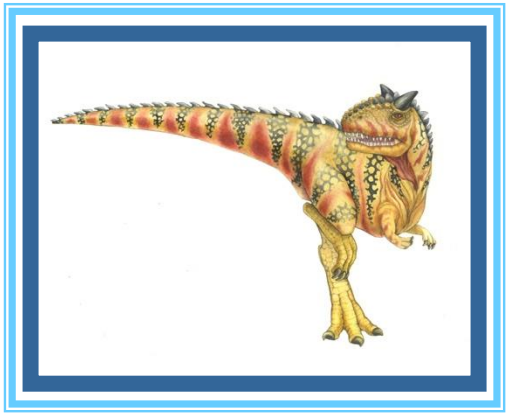
Fire Alarm/General Emergency

If you receive an OU Alert that there is danger inside or near the building, or the fire alarm inside the building activates:

1. *LEAVE* the building. Do not use the elevators.
2. *KNOW* at least two building exits
3. *ASSIST* those that may need help
4. *PROCEED* to the emergency assembly area
5. *ONCE safely outside, NOTIFY first responders of anyone that may still be inside building due to mobility issues.*
6. *WAIT* for official notice before attempting to re-enter the building.

OU Fire Safety on Campus

Operating System Overview – Part 1



The slides are adopted from the textbook chapter 1

Silberschatz, Galvin and Gagne ©2018



After this session you should be able to:

- recall the computer system structure
- recall the computer system organization
- explain what an operating system (OS) does
- explain what kernel is and describe its main functions





In small group (2 or 3 people), discuss the following:

- What is an operating system (OS)?
- What are some examples of OS?
- Why do we need an OS?
- What types of devices use an OS?





Introduction

- What is an Operating System?
  - Have you ever used one? Examples?

















Why we need an OS?

- To interact with machines



OS







What types of devices use an OS?













What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner





Computer System Structure

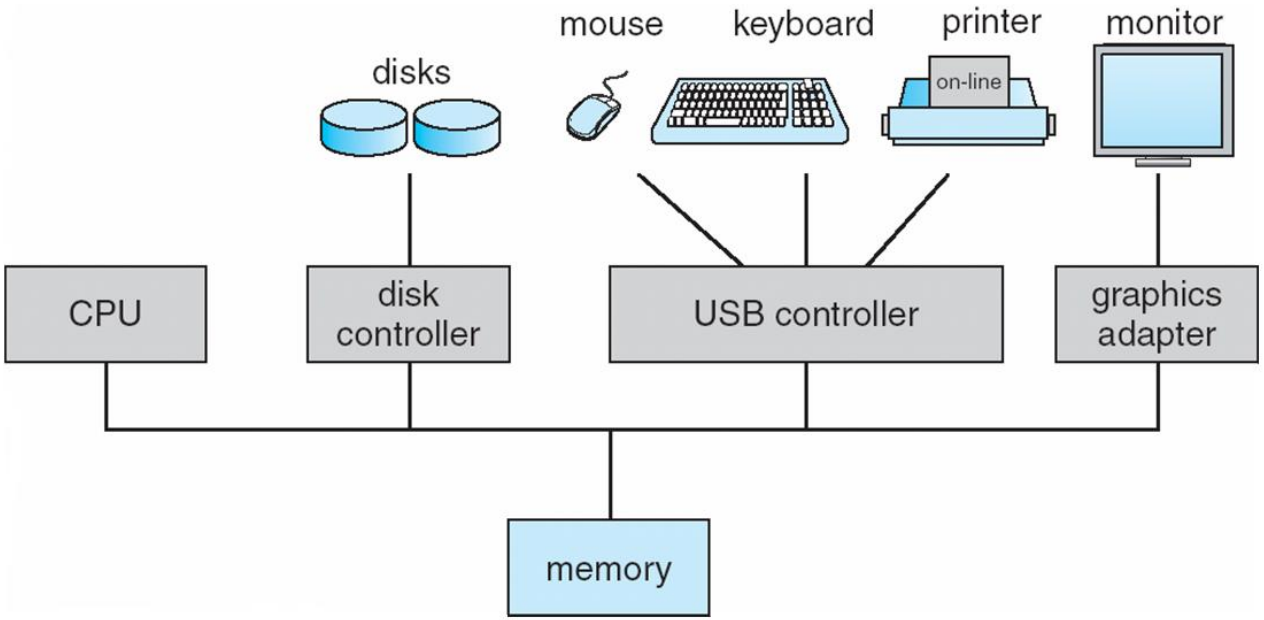
- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers





Computer System Organization

- Computer-system operation
  - One or more CPUs, device controllers connect through common bus providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles

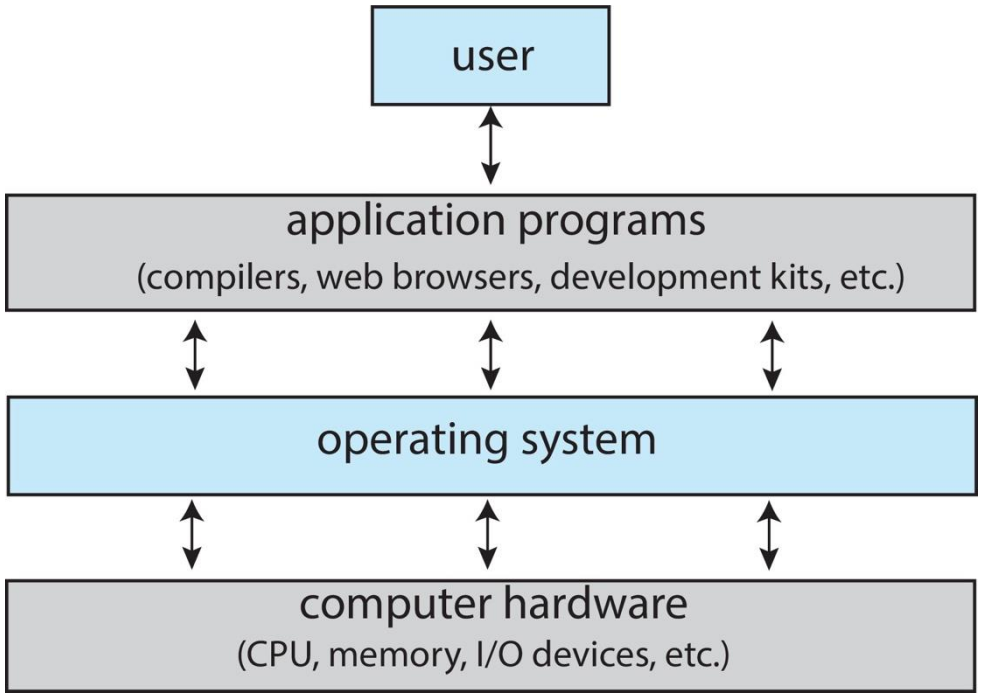


The Hardware





Abstract View of Components of Computer







What Operating Systems Do

- **Depends on the point of view**
  - Users want convenience, **ease of use** and **good performance**
    - Don't care about **resource utilization**
  - But shared computer such as **mainframe** or **minicomputer** must keep all users happy
  - Users of dedicate systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
  - Handheld computers are resource poor, optimized for usability and battery life
  - Some computers have little or no user interface, such as embedded computers in devices and automobiles





Operating System Definition

- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer





Operating System Definition (Cont.)

- No universally accepted definition
- "Everything a vendor ships when you order an operating system" is a good approximation
  - But varies wildly
- "The one program running at all times on the computer" is the **kernel**.
- Everything else is either
  - a system program (ships with the operating system) , or
  - an application program.





Computer Startup

- **bootstrap program** is loaded at power-up or reboot
  - Typically stored in ROM or EPROM, generally known as **firmware**
  - Initializes all aspects of system
  - Loads operating system kernel and starts execution





Kernel's Primary Functions

- **Resource management:**
  - Memory management → allocating memory to processes as needed
  - CPU scheduling → determining which processes get to use CPU and for how long (fair and efficient use of processor)
- **Process management:**
  - Process creation and termination
  - Process isolation (preventing processes from interfering with each other)
  - Inter process communication (IPC)

process = running program





Kernel's Primary Functions (Cont.)

- **Device Management**
  - Device drivers → translator between the OS and the hardware devices
  - Input/Output operations → ensuring data are correctly read from and written to the devices
- **File System Management**
  - Handling reading and writing data to storage devices
  - Managing file systems
  - Controlling access to files and directories
- **Security and Access Control**
  - Managing user authentication and permission
  - Restrict access to resources based on user permission



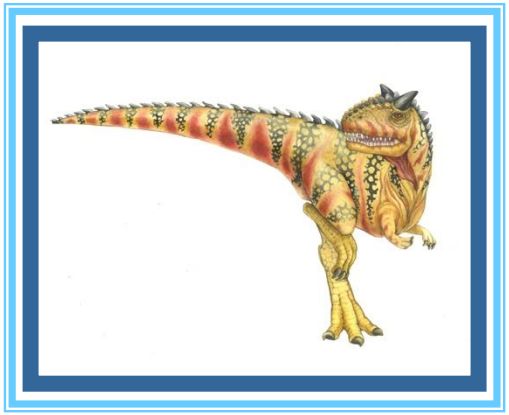


Kernel's Primary Functions (Cont.)

- **System Calls Handling**
  - Provides an interface for applications to request services from OS through system calls such as:
    - 'fork()', 'exec()', '(exit)' → process control
    - 'open()', 'read()', 'write()', 'close()' → file management
    - 'getpid()' → information about system data
    - 'send()', 'recv()' → process or network communication
- **Interrupt Handling**



Topic 1: OS Overview Part 2



Silberschatz, Galvin and Gagne ©2018



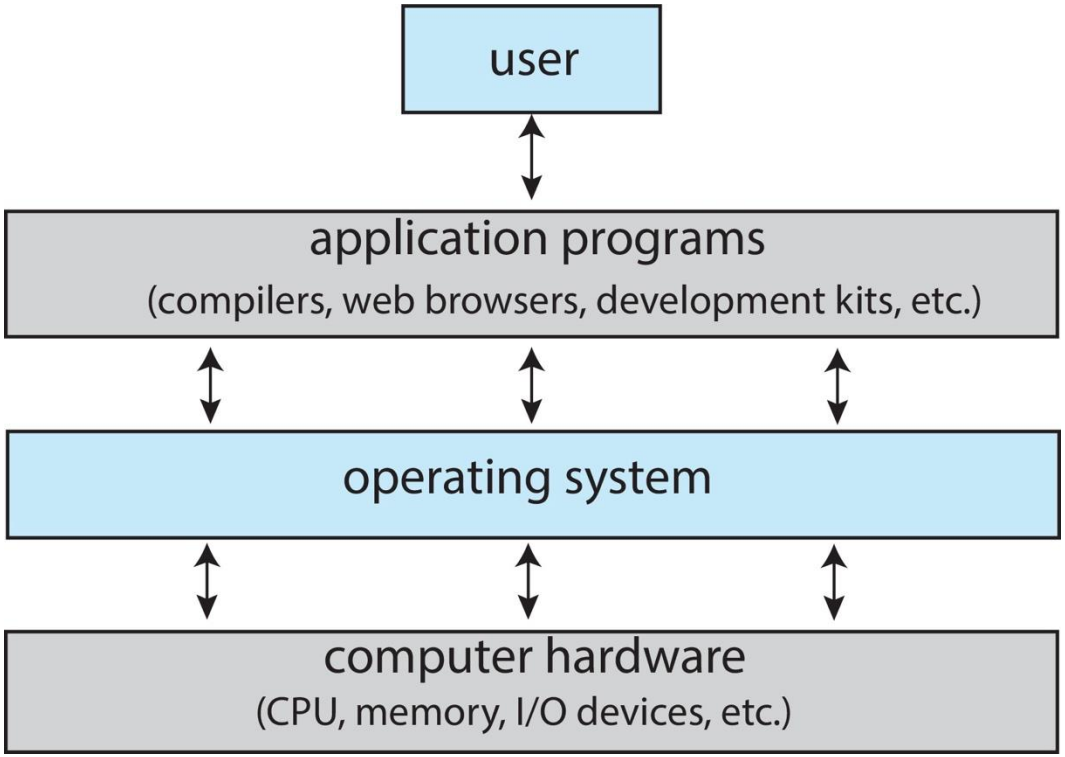
After this session you should be able to:

- Explain why an operating system is described as **interrupt-driven**.
- Describe the **different types of interrupts**.
- Distinguish between **user mode and kernel mode**.
- Explain what **system calls** are and why they are used.





Abstract View of Components of Computer



OS: controls the hardware and coordinates its use among various applications programs (for various user*)





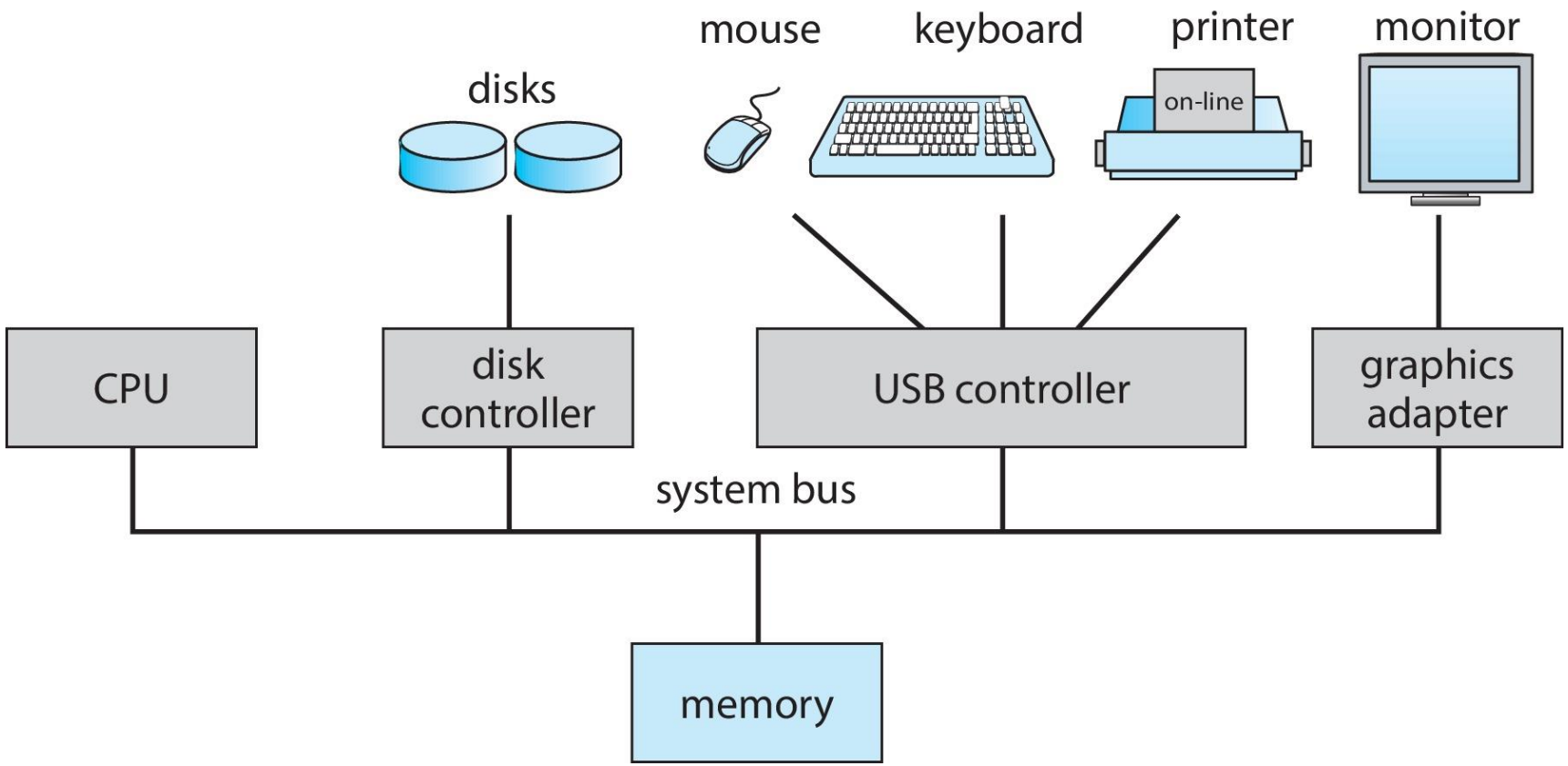
Operating System Definition

- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer





Computer System Organization







CPU

- CPU designed to perform simple operations on pieces of data
  - Examples: reading data, adding, subtracting, multiplying, dividing numbers
  - Understands instructions written in machine language and included in its instruction set
    - Each brand of CPU has its own instruction set
- To carry out meaningful calculation, CPU must perform many operations

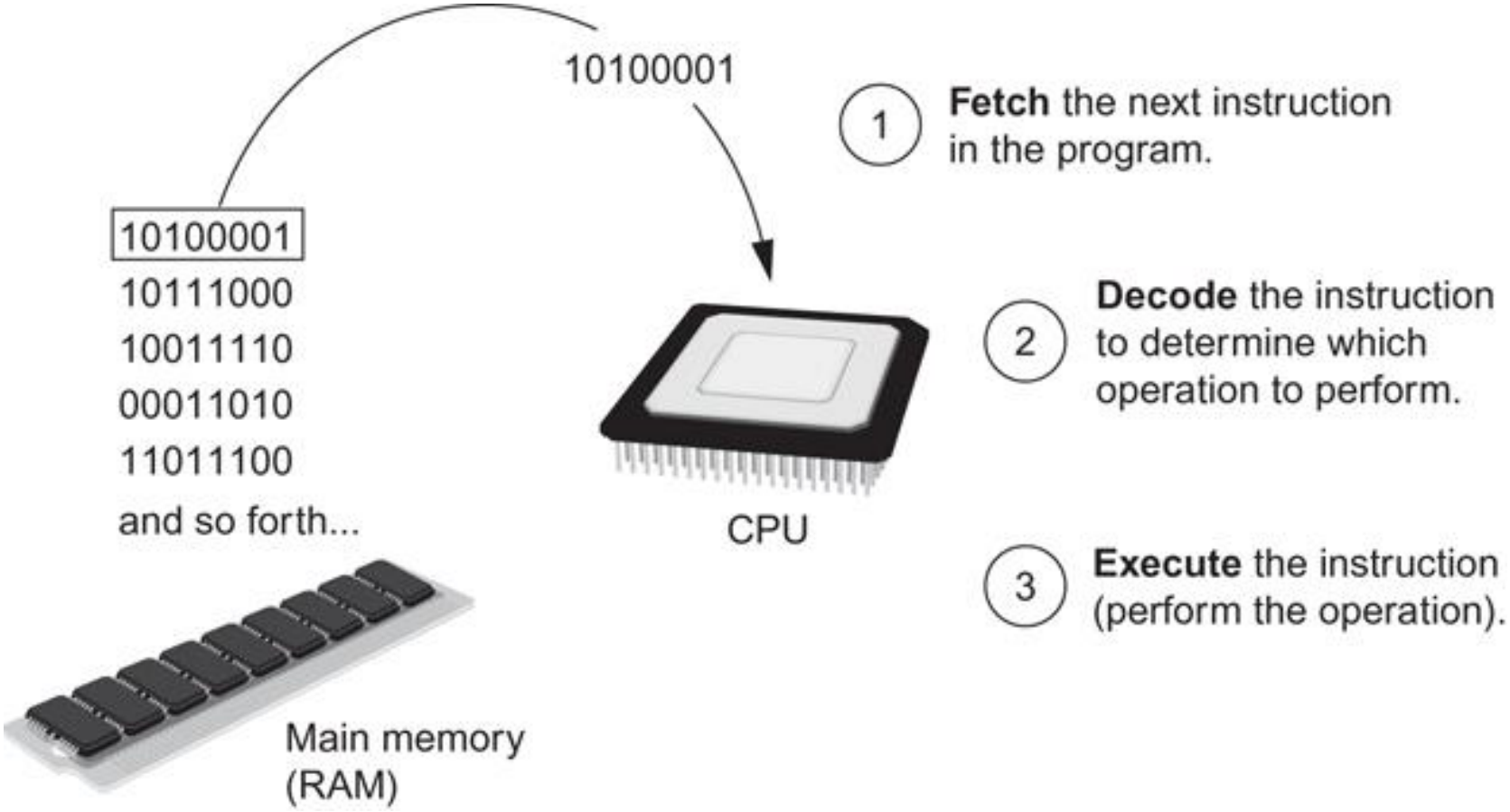
- The CPU has an **internal clock** **(clock cycle)** that ticks at a regular rate.
- A CPU clock cycle is the smallest increment of time in which a processor can perform an action.

Hertz

What does it mean by CPU having a 1GHz speed?



How a Running Program (Process)Works



CPU relies on the program counter to execute the next instruction

Let’s consider the following program

add.c

```c
#include <stdio.h>

int main() {
    int a = 5;
    int b = 3;
    int sum = a + b;    // User-mode computation
    printf("The sum is: %d\n", sum);  // Requires OS support
    return 0;
}
```

topic1 — -zsh — 80×24

```
(base) egapanjei@ega topic1 % ls
add.c
(base) egapanjei@ega topic1 % gcc add.c -o add.out
(base) egapanjei@ega topic1 % ls
add.c   add.out
(base) egapanjei@ega topic1 % ./add.out
The sum is: 8
(base) egapanjei@ega topic1 %
```



Example of instructions

```
int sum = a + b;
```

	x86-64 Assembly (Intel/AMD)	What the CPU is doing
1	`mov eax, [rbp - 4]`	Load the a from memory address [rbp -4] into a CPU register (eax)
2	`add eax, [rbp - 8]`	CPU loads the value of b at memory location [rbp - 8], adds it to the current value in eax
3	`mov [rbp - 12], eax`	Store the value in the eax register into the memory address [rbp -12]



Now consider the following scenario:

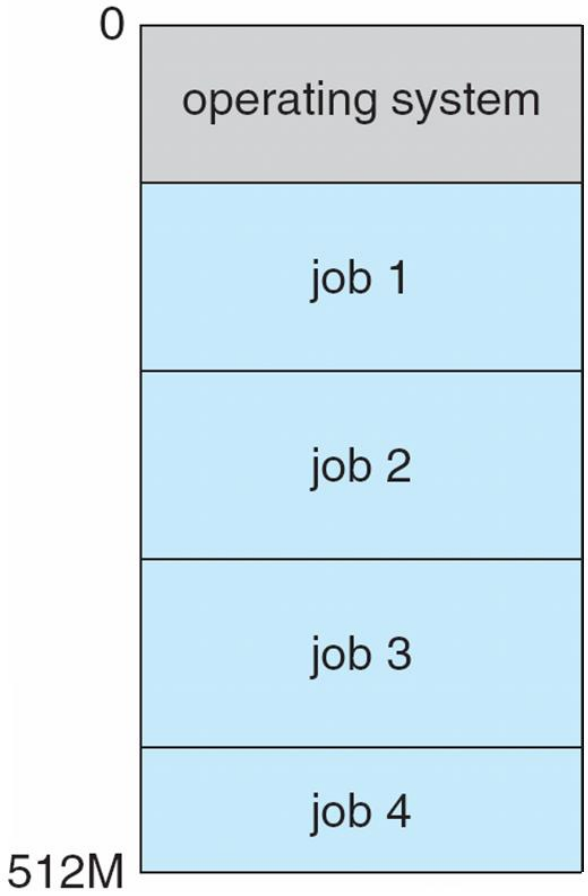
- Suppose our computer only has one core CPU
- 3 programs are running:
  - process1: open and read file1.txt from the disk and show (print) the content of the file
  - process2: open and write some content to file2.txt
  - process3: perform some computation based on the user input

In a group of 2 or 3 people, discuss:

- **Is it possible for the CPU to execute instruction of process1, process2, process3 simultaneously?**
- **How can all the three processes running at the same time?**



Memory Layout for Multiprogrammed System

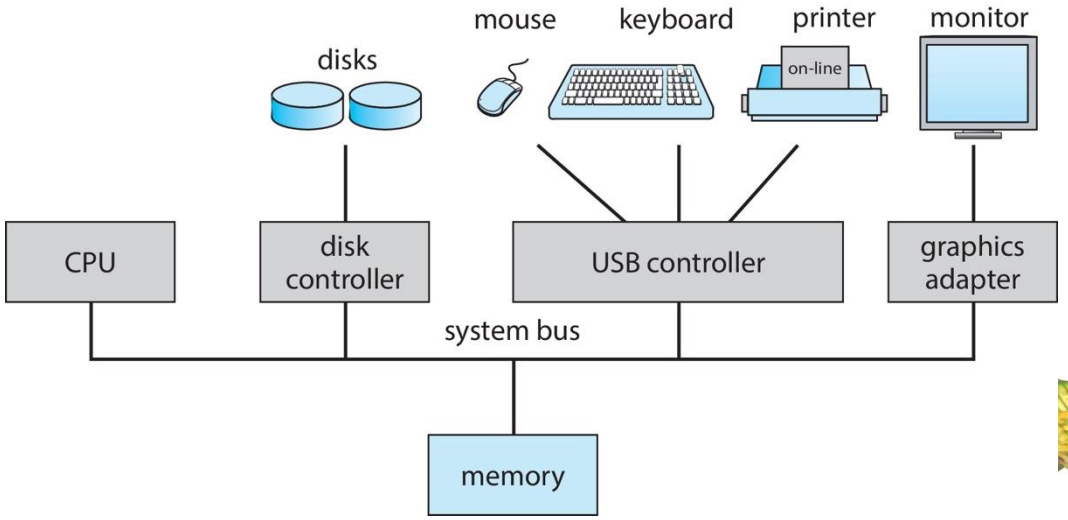






Computer-System Operation

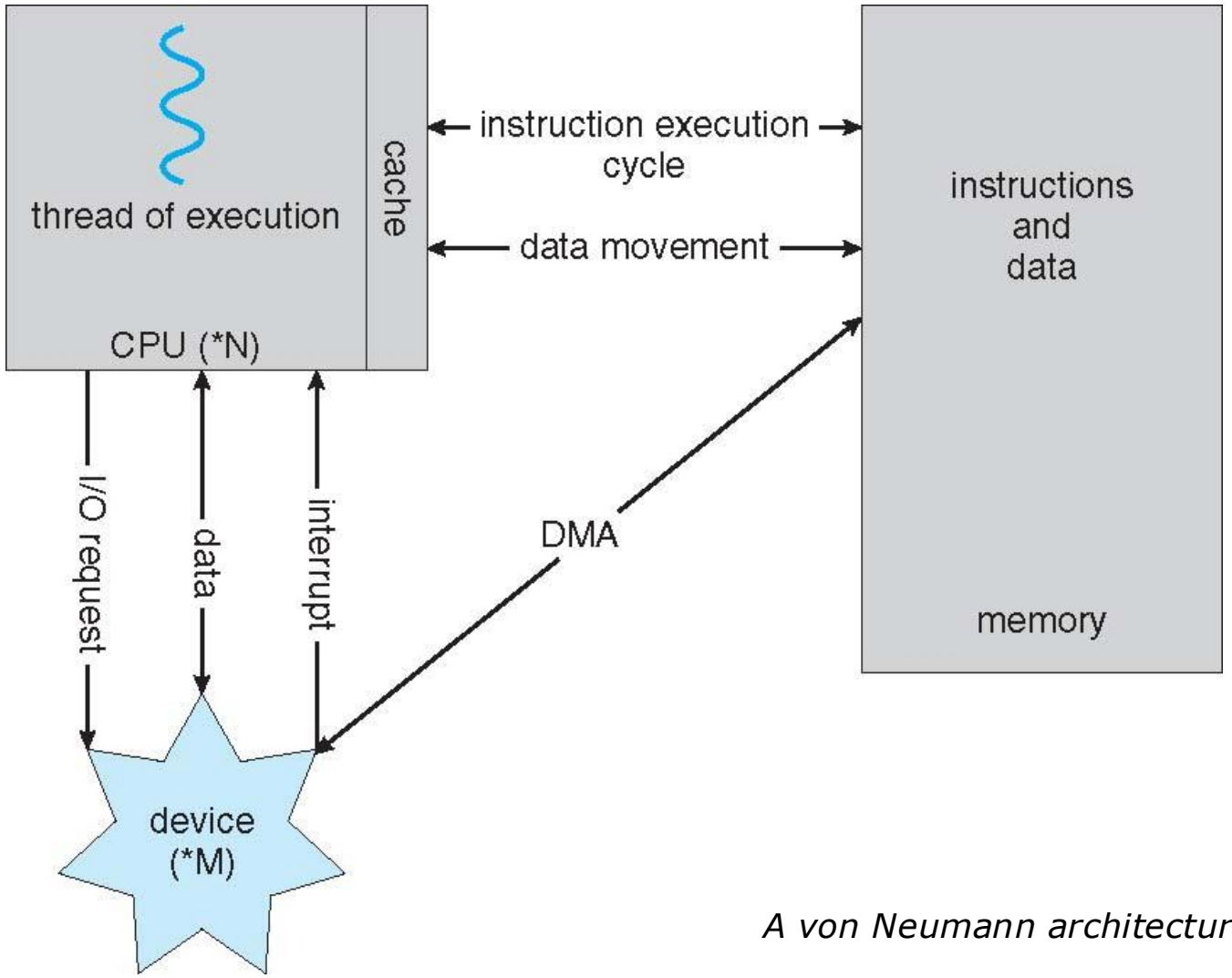
- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- *CPU moves data from/to main memory to/from local buffers*
  **** the old way*
- I/O is from the device to local buffer of controller
  - Device controller informs CPU that it has finished its operation by causing an interrupt
  - The OS handles these interrupts to coordinate I/O.



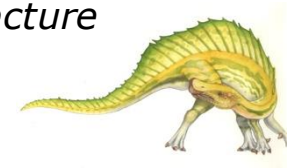




How a Modern Computer Works



A von Neumann architecture





Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte





Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven**





Interrupt Handling

- Anytime any kind of interrupt takes place, the OS kernel is guaranteed to take over the CPU time.
- The operating system preserves the state of the CPU by storing registers and the program counter
- Determines which type of interrupt has occurred:
  - **polling (the old/simple way)**
  - **vectored** interrupt system
- Separate segments of code determine what action should be taken for each type of interrupt





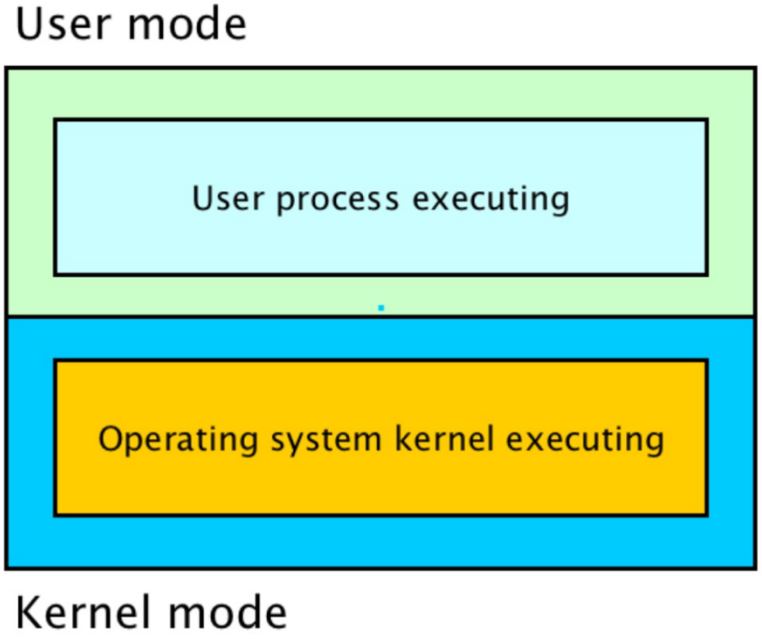
- Let's summarize the types of interrupts we have seen so far:





Dual-mode Operation

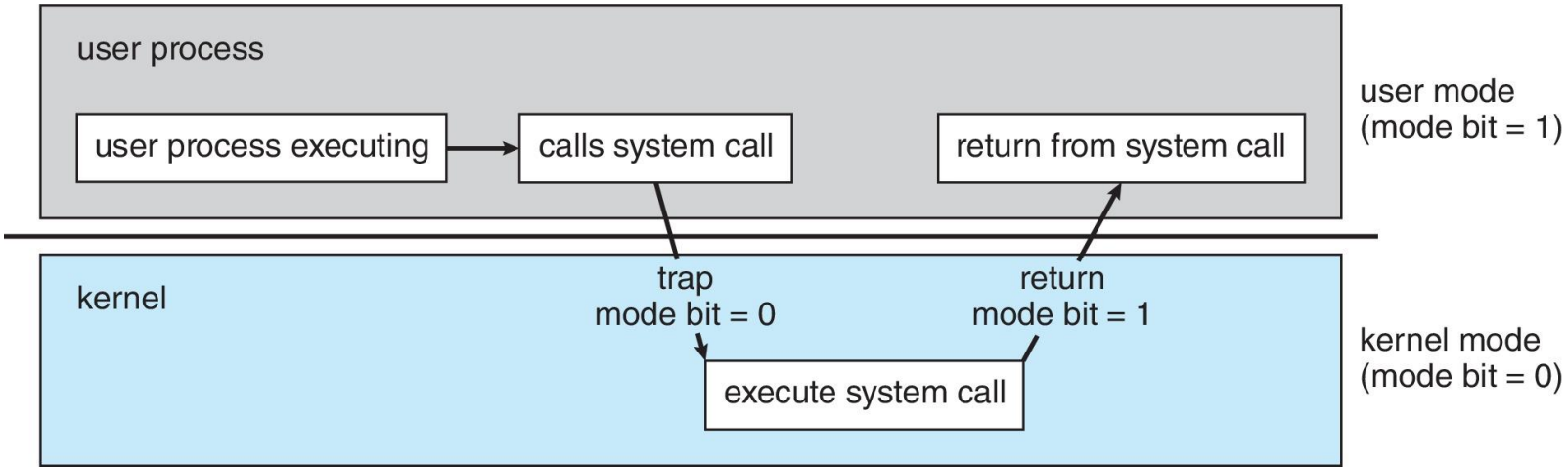
- **Dual-mode** operation allows OS to protect itself and other system components:
  - **User mode** and **kernel mode**
- **Mode bit** provided by hardware
  - User mode: mode bit = 1
  - Kernel mode: mode bit = 0







Transition from User to Kernel Mode



- How do we guarantee that user does not explicitly set the mode bit to “kernel”?
  - System call changes mode to kernel, return from call resets it to user
- Some instructions designated as **privileged**, only executable in kernel mode





Transition from User to Kernel Mode

- We must ensure that the operating system maintains control over the CPU.

- Timer to prevent infinite loop / process hogging resources
  - Timer is set to interrupt the computer after some time period
  - Keep a counter that is decremented by the physical clock.
  - Operating system set the counter (privileged instruction)
  - When counter zero generate an interrupt
  - Set up before scheduling process to regain control or terminate program that exceeds allotted time





- Let's summarize the types of interrupts we have seen so far:





Back to our simple program

add.c

```c
#include <stdio.h>

int main() {
    int a = 5;
    int b = 3;
    int sum = a + b;   // User-mode computation
    printf("The sum is: %d\n", sum);  // Requires OS support
    return 0;
}
```

Which line trigger the change from user mode to kernel mode?





System Calls

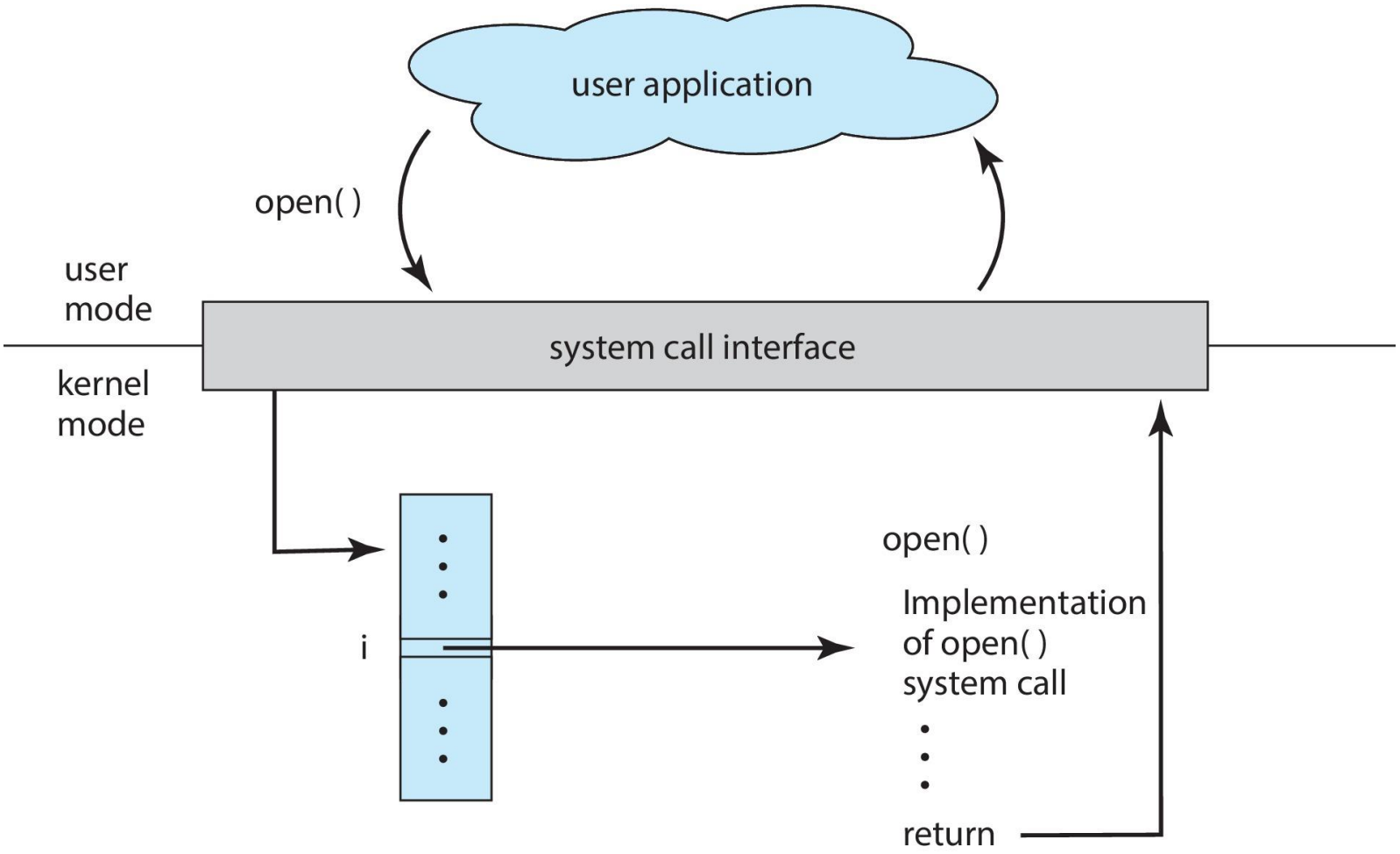
- Programming interface to the services provided by the OS
- Typically written in a high-level language (e.g. C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Note that the system-call names used throughout this text are generic





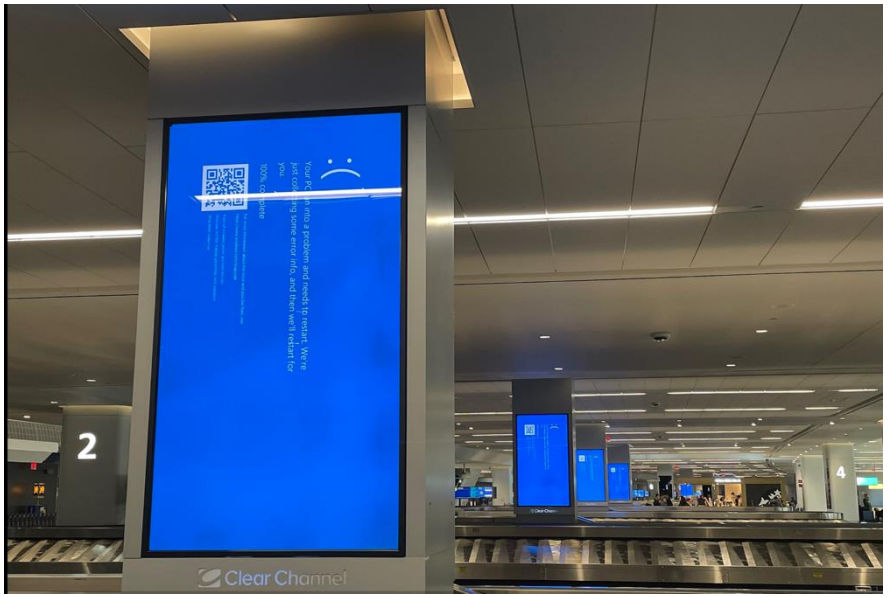
API – System Call – OS Relationship







Some interesting videos



CloudStrike case
https://www.youtube.com/watch?v=wAzEJxOo1ts

Most popular operating systems
https://www.youtube.com/watch?v=Tjc-qrjzRcU





The next three slides cover additional details on Computer System Architecture.





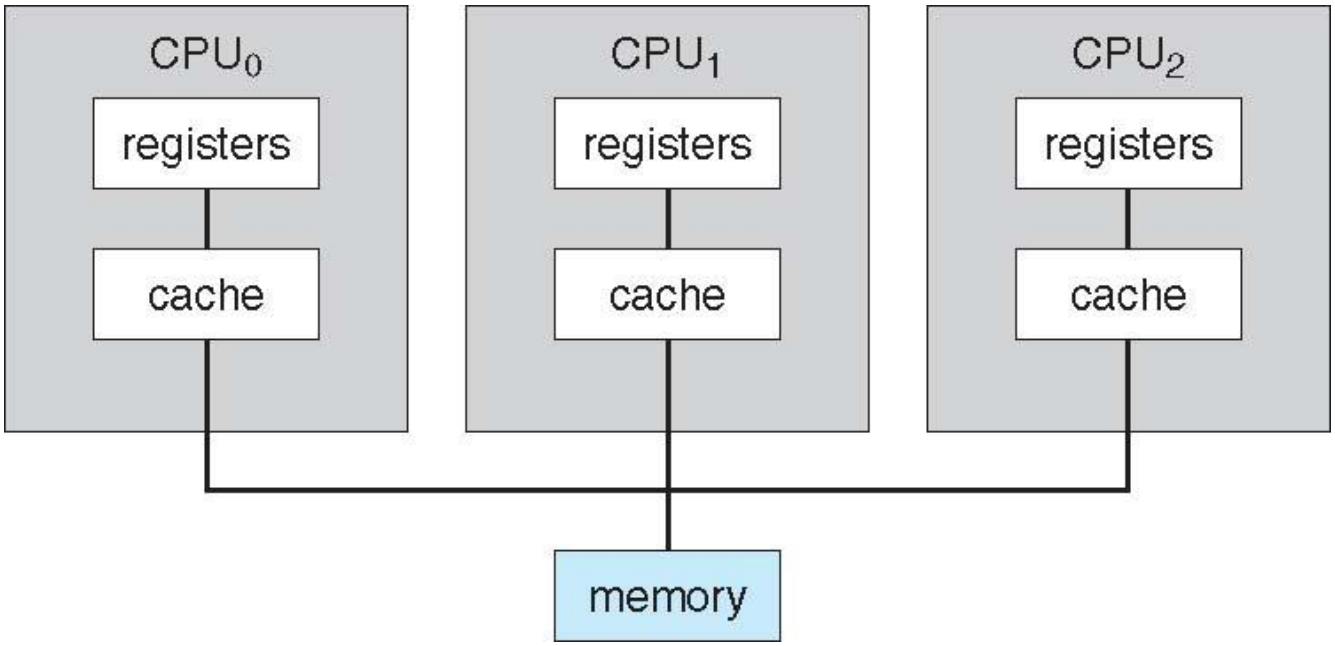
Computer-System Architecture

- Most systems use a single general-purpose processor
  - Most systems have special-purpose processors as well
- **Multiprocessors** systems growing in use and importance
  - Also known as **parallel systems**, **tightly-coupled systems**
  - Advantages include:
    1. **Increased throughput**
    2. **Economy of scale**
    3. **Increased reliability** – graceful degradation or fault tolerance
  - Two types:
    1. **Asymmetric Multiprocessing** – each processor is assigned a specie task.
    2. **Symmetric Multiprocessing** – each processor performs all tasks





Symmetric Multiprocessing Architecture

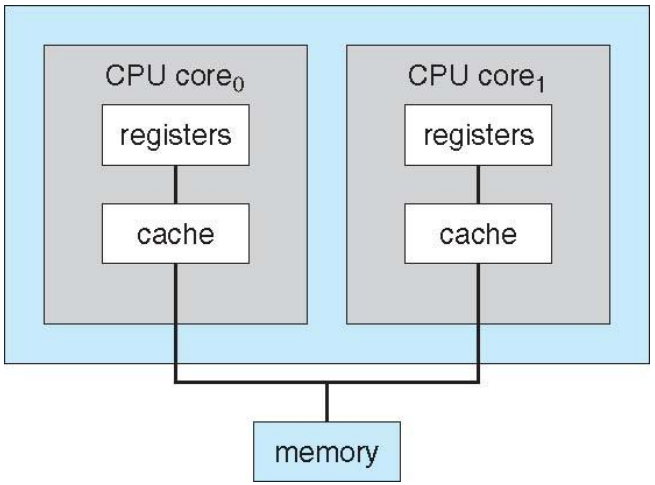






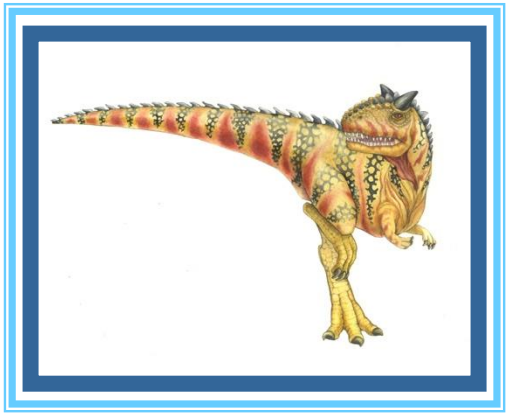
A Dual-Core Design

- Multi-chip and **multicore**
- Systems containing all chips
  - Chassis containing multiple separate systems





Topic 1: OS Overview Part 3



The materials in this session are from the textbook Chapter 2

Silberschatz, Galvin and Gagne ©2018



After completing this session, you should be able to:

- Explain the purpose of system calls
- Describe the roles of a linker and a loader and illustrate how they handle program code during compilation and execution.
- Explain why applications may be operating system specific, citing at least one example.
- Differentiate between policy and mechanism in OS design, giving a concrete example of each.
- Compare and contrast different operating system structures (monolithic, layered, microkernel, loadable kernel modules) and identify one advantage and disadvantage of each.





Let's recap the types of interrupt

- **OS is interrupt driven.**
- What are the types of system interrupts that help the OS manage hardware and software tasks?





Group Discussion

- In a group of 2 or 3 people discuss the following:

program1.c

```c
#include <stdio.h>   // Include standard I/O library for printf and scanf
#include <stdlib.h>  // Include standard library for exit and other functions

int main() {         // Start of the main function
    int a = 5;       // Declare an integer variable 'a' and assign 5
    int b = 10;      // Declare an integer variable 'b' and assign 10
    int sum = a + b; // Compute the sum of 'a' and 'b' and store in 'sum'

    printf("The sum is: %d\n", sum);  // Print the value of 'sum' to the console

    char name[50];   // Declare a character array to store the user's name

    printf("Enter your name: "); // Prompt the user to enter their name
    scanf("%s", name);           // Read a string from the user's input (keyboard)

    printf("Hello, %s!\n", name); // Greet the user by name

    return 0;        // End of program, return 0 to indicate success
}
```

When the program is run, which lines of the main function are executed in user mode and which ones in kernel mode?





System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Note that the system-call names used throughout this text are generic





Example of System Calls (1)

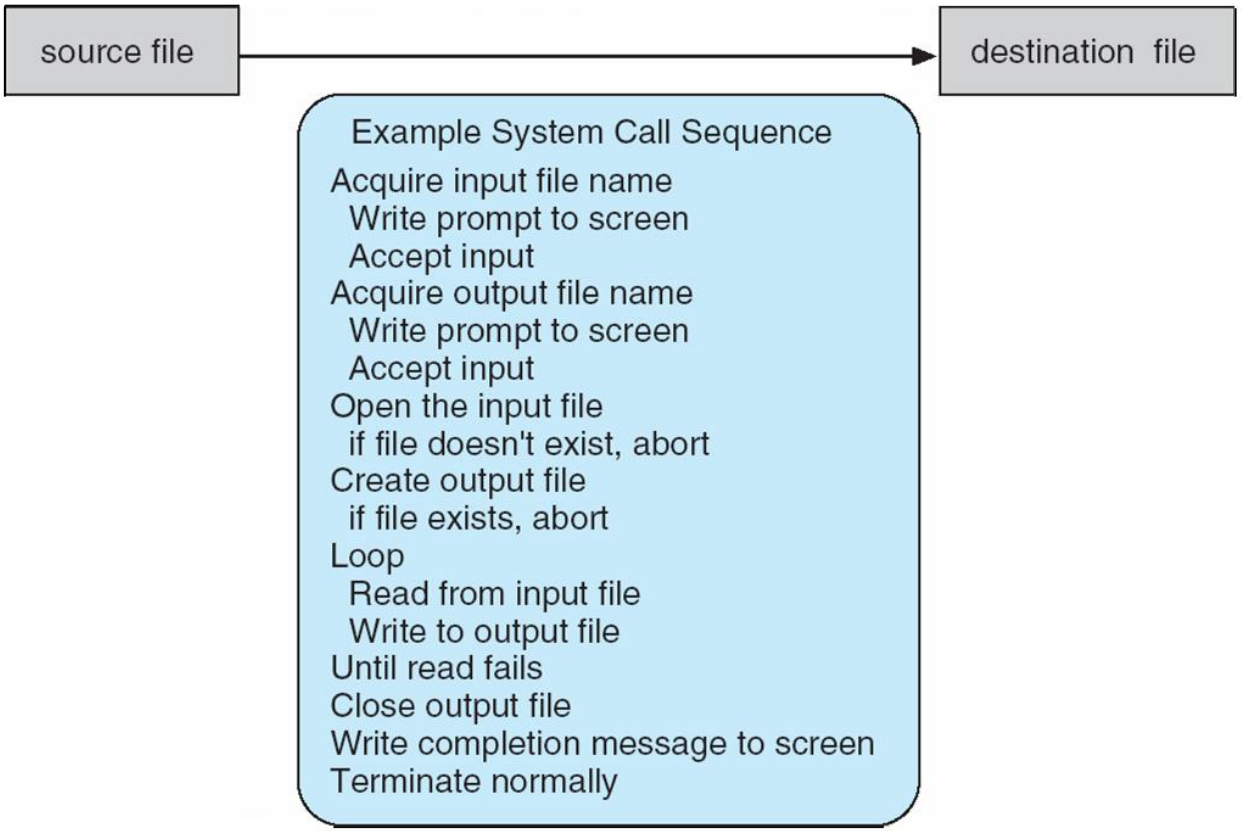
- Suppose you want to copy a file to another file
- **cp in.txt out.txt**





Example of System Calls (2)

- System call sequence to copy the contents of one file to another file



`Check program2.c`





Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns –1.





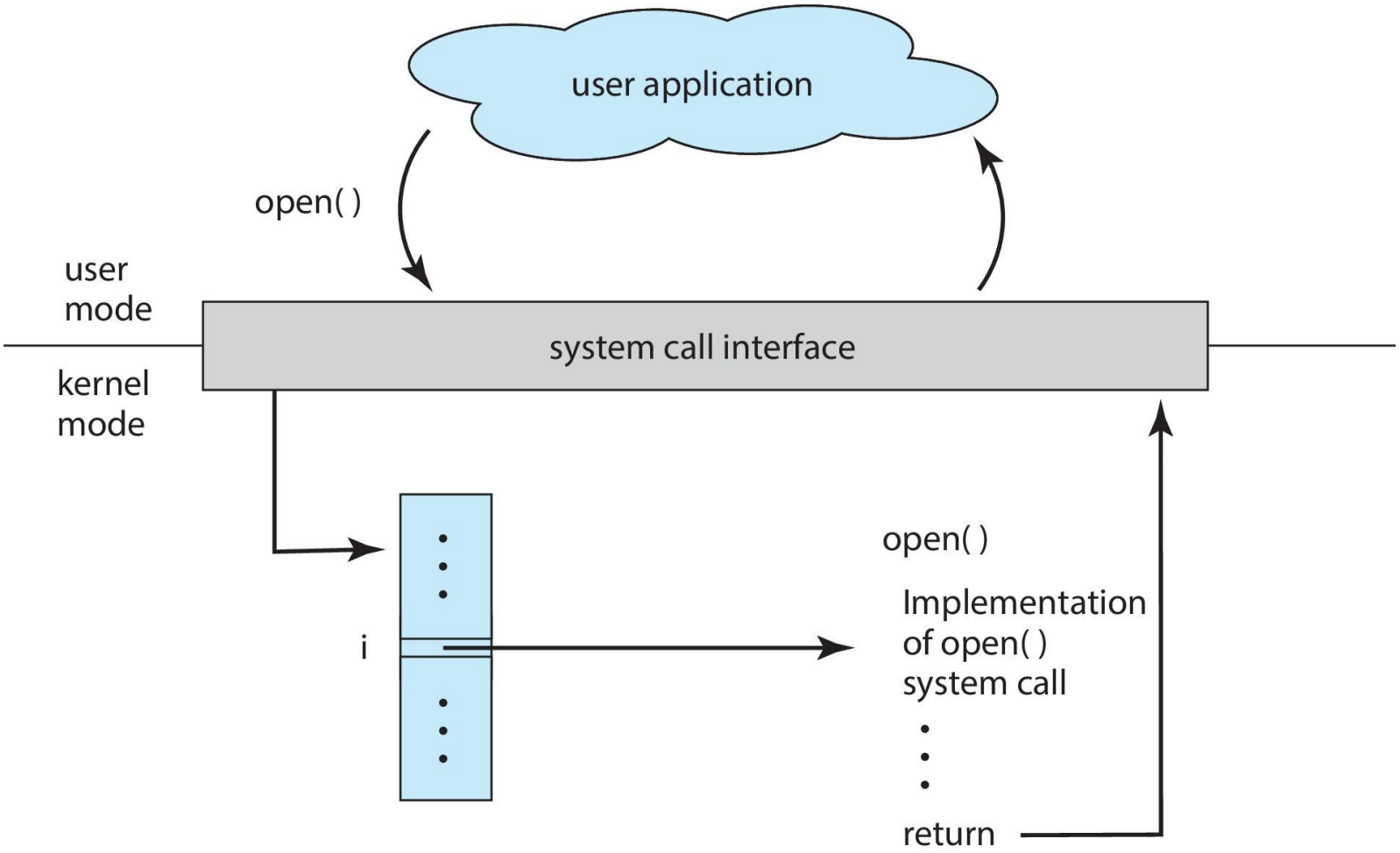
System Call Implementation

- Typically, a number associated with each system call
  - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)





API – System Call – OS Relationship







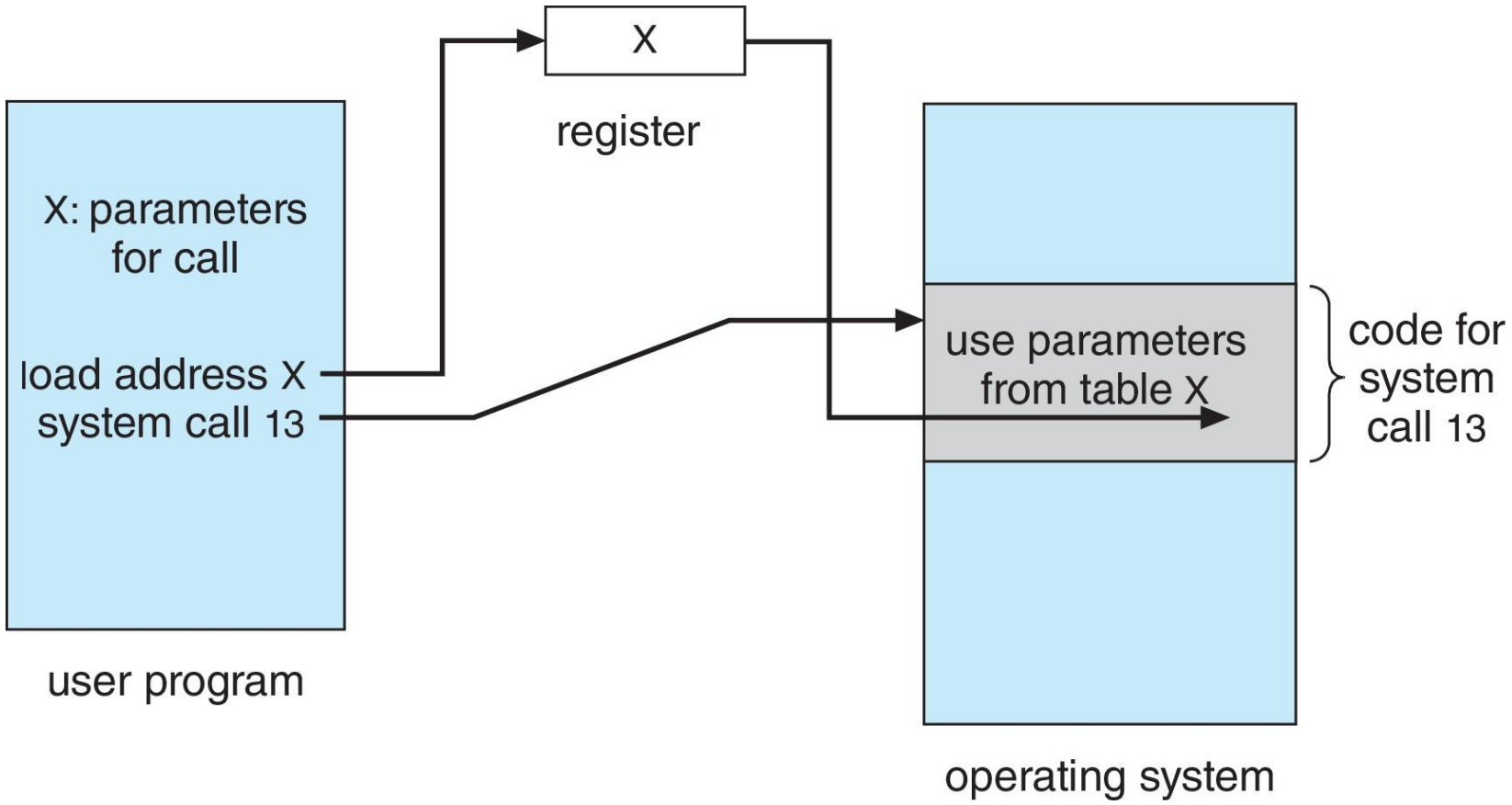
System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
  - Exact type and amount of information vary according to OS and call
- **Three general methods** used to pass parameters to the OS
  1. Simplest: pass the parameters in **registers**
     - In some cases, may be more parameters than registers
  2. Parameters stored in a **block**, or table, in memory, and address of block passed as a parameter in a register
     - Linux uses a combination of these approaches 1 & 2
  3. Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed





Parameter Passing via Table







Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitlializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`



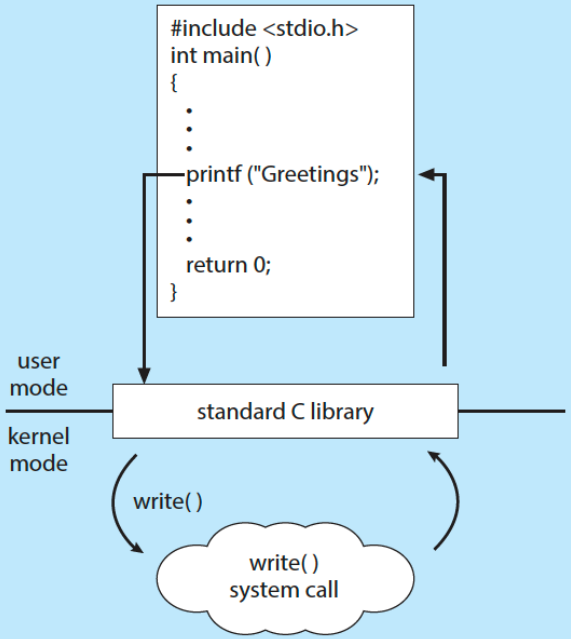


Standard C Library Example

- C program invoking printf() library call, which calls write() system call

THE STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:







System Services

- ***System services***, also known as ***system utilities***, provide a convenient environment for program development and execution.
- They can be divided into:
  - File manipulation
  - Status information sometimes stored in a file
  - Programming language support
  - Program loading and execution
  - Communications
  - Background services
  - Application programs
- The view of the operating system seen by most users is defined by the application and system programs, rather than by the actual system calls.





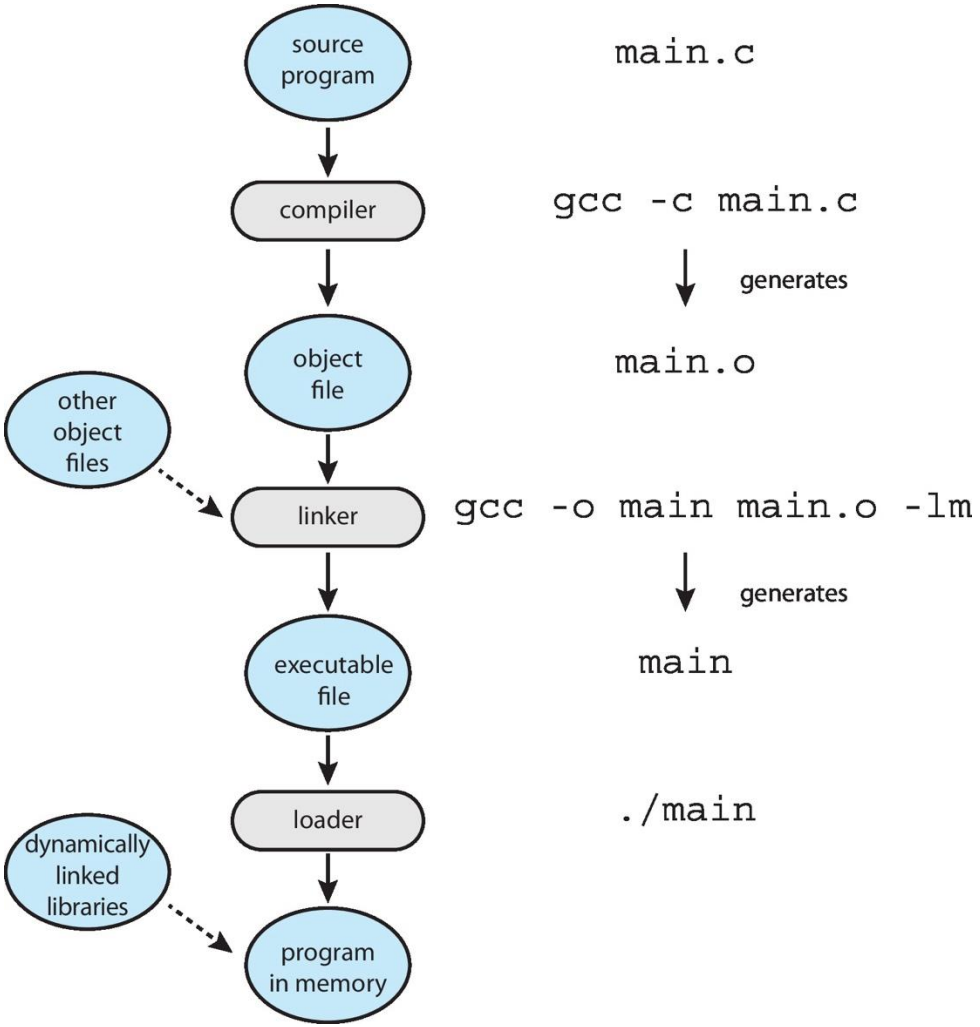
Linkers and Loaders

- Source code compiled into object files designed to be loaded into any physical memory location – **relocatable object file**
- **Linker** combines these into single binary **executable** file
  - Also brings in libraries
- Program resides on secondary storage as binary executable
- Must be brought into memory by **loader** to be executed
  - **Relocation** assigns final addresses to program parts and adjusts code and data in program to match those addresses
- Modern general purpose systems don't link libraries into executables
  - Rather, **dynamically linked libraries** (in Windows, **DLLs**) are loaded as needed, shared by all that use the same version of that same library (loaded once)
- Object, executable files have standard formats, so operating system knows how to load and start them





The Role of the Linker and Loader







Why Applications are Operating System Specific

- Apps compiled on one system usually not executable on other operating systems
- Each operating system provides its own unique system calls
  - Own file formats, etc
- Apps can be multi-operating system
  - Written in interpreted language like Python, Ruby, and interpreter available on multiple operating systems
  - App written in language that includes a VM containing the running app (like Java)
  - Use standard language (like C), compile separately on each operating system to run on each
- **Application Binary Interface** (**ABI**) is architecture equivalent of API, defines how different components of binary code can interface for a given operating system on a given architecture, CPU, etc





Operating System Design and Implementation

- Design and Implementation of OS not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start the design by defining goals and specifications
- Affected by choice of hardware, type of system
- **User** goals and **System** goals
  - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
  - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient





Operating System Design and Implementation (Cont.)

- Important principle to separate

  Policy: ***What*** will be done?
  Mechanism: ***How*** to do it?

- Mechanisms determine how to do something, policies decide what will be done
- The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later (example – timer)
- Specifying and designing an OS is highly creative task of **software engineering**





Implementation

- Much variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Actually usually a mix of languages
  - Lowest levels in assembly
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
  - But slower
- **Emulation** can allow an OS to run on non-native hardware





Operating System Structure

- General-purpose OS is very large program
- Various ways to structure ones
  - Simple structure – MS-DOS
  - More complex -- UNIX
  - Layered – an abstrcation
  - Microkernel -Mach





Monolithic Structure – Original UNIX

UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

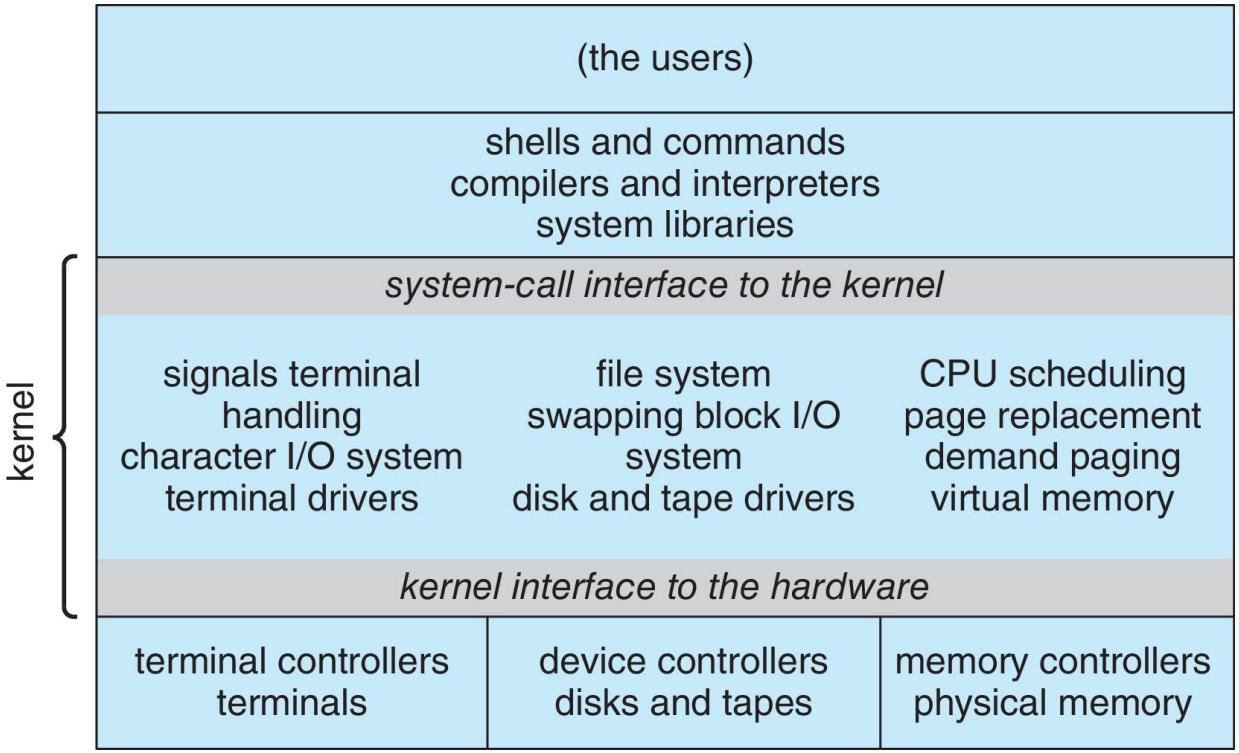
- Systems programs
- The kernel
  - Consists of everything below the system-call interface and above the physical hardware
  - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level





Traditional UNIX System Structure

Beyond simple but not fully layered

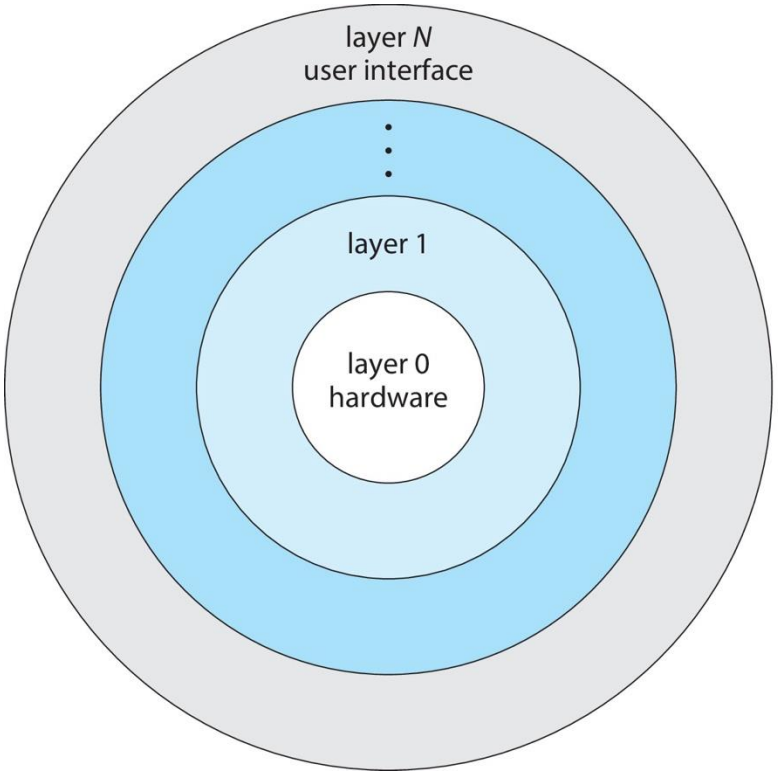






Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers







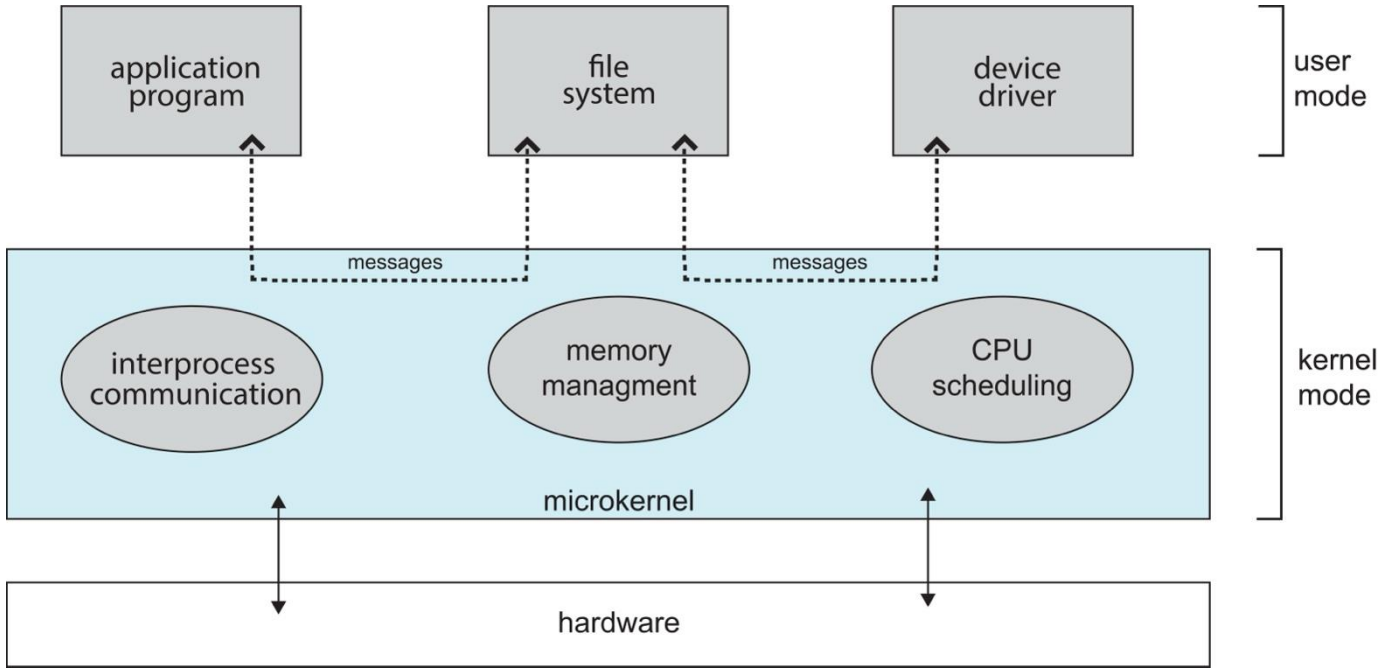
Microkernels

- Moves as much from the kernel into user space
- **Mach** example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication





Microkernel System Structure







Modules

- Many modern operating systems implement **loadable kernel modules** (**LKMs**)
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
  - Linux, Solaris, etc





- Review the following slides, which illustrate the Hybrid operating system structure in Linux, macOS, iOS, and Android.
- Please go through them at your own pace.





Hybrid Systems

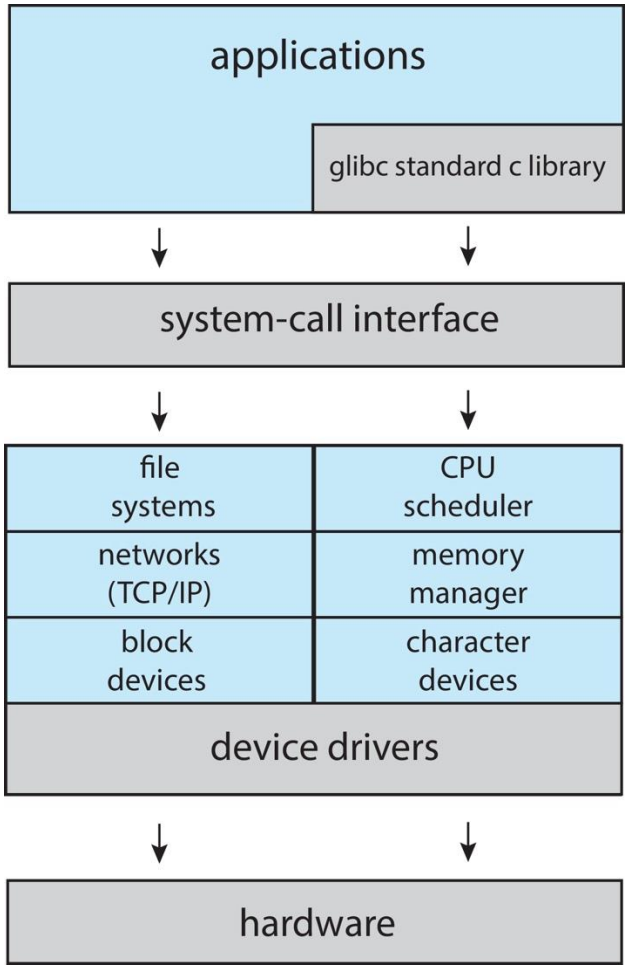
- Most modern operating systems are actually not one pure model
  - Hybrid combines multiple approaches to address performance, security, usability needs
  - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
  - Windows mostly monolithic, plus microkernel for different subsystem ***personalities***
- Apple Mac OS X hybrid, layered, **Aqua** UI plus **Cocoa** programming environment
  - Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called **kernel extensions**)





Linux System Structure

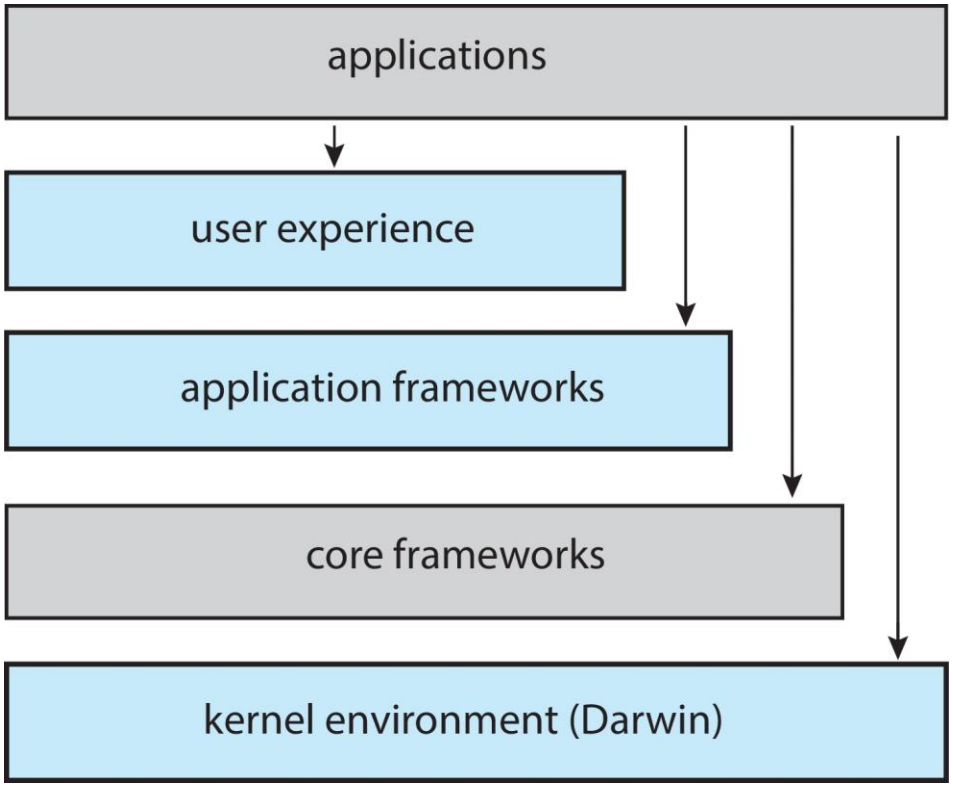
Monolithic plus modular design







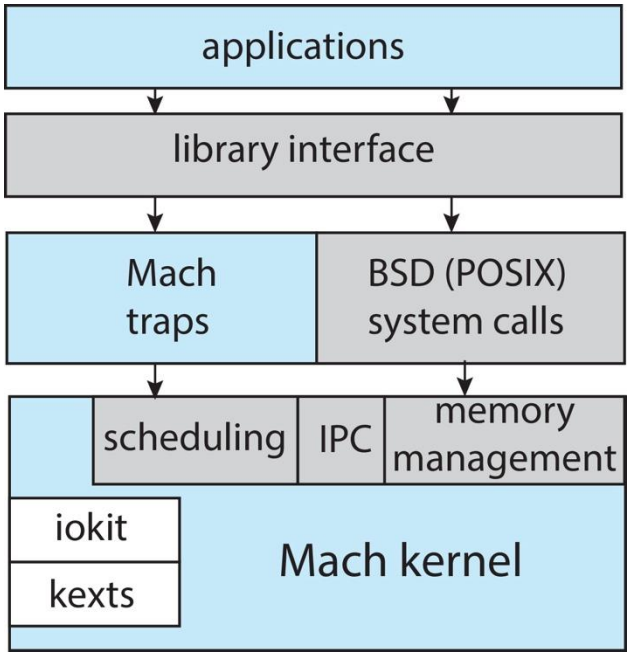
macOS and iOS Structure







Darwin


applications
library interface
Mach traps
BSD (POSIX) system calls
scheduling
IPC
memory management
iokit
kexts
Mach kernel





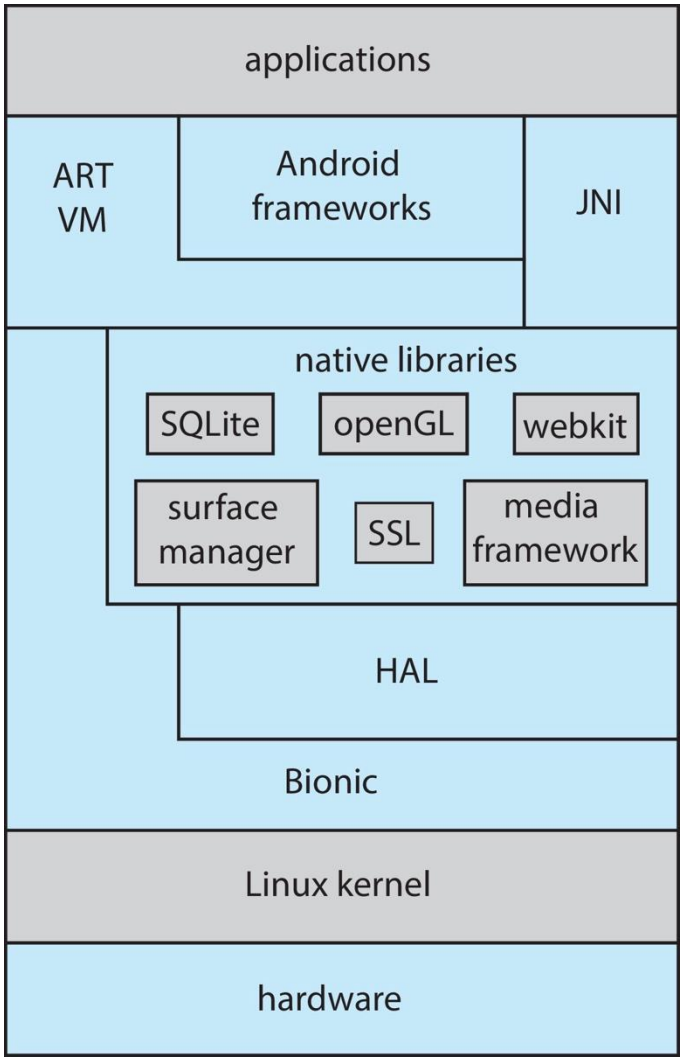
Android

- Developed by Open Handset Alliance (mostly Google)
  - Open Source
- Similar stack to IOS
- Based on Linux kernel but modified
  - Provides process, memory, device-driver management
  - Adds power management
- Runtime environment includes core set of libraries and Dalvik virtual machine
  - Apps developed in Java plus Android API
    - Java class files compiled to Java bytecode then translated to executable than runs in Dalvik VM
- Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc



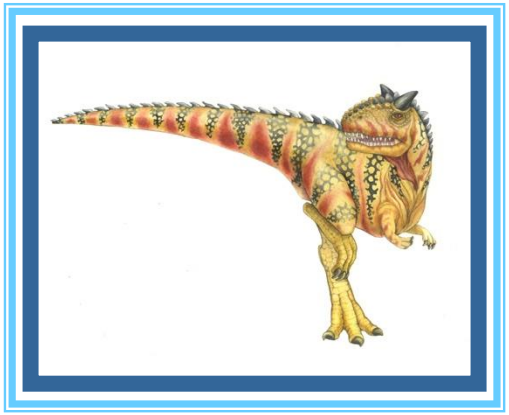


Android Architecture





Topic 2: Process Part 1



The content of this session is from the textbook Chapter 3

Silberschatz, Galvin and Gagne ©2018



After this session, you should be able to:

- Draw and label the typical virtual memory layout of a process.
- Identify which parts of the process belong to: text, data, heap, and stack sections.
- Describe the five-state process model diagram (New → Ready → Running → Waiting → Terminated) and explain each transition event/condition.
- Describe a context switch and explain the sources of overhead that occur during a context switch.
- List at least 5 major categories of information stored in the Process Control Block (PCB) and briefly state the purpose of each.
- Identify and describe the operating system data structures used for process scheduling.
- Explain and differentiate the types of process scheduling used by an operating system.





Process Concept

- An operating system executes a variety of programs:
  - Batch system – **jobs**
  - Time-shared systems – **user programs** or **tasks**
- Textbook uses the terms ***job*** and ***process*** almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion





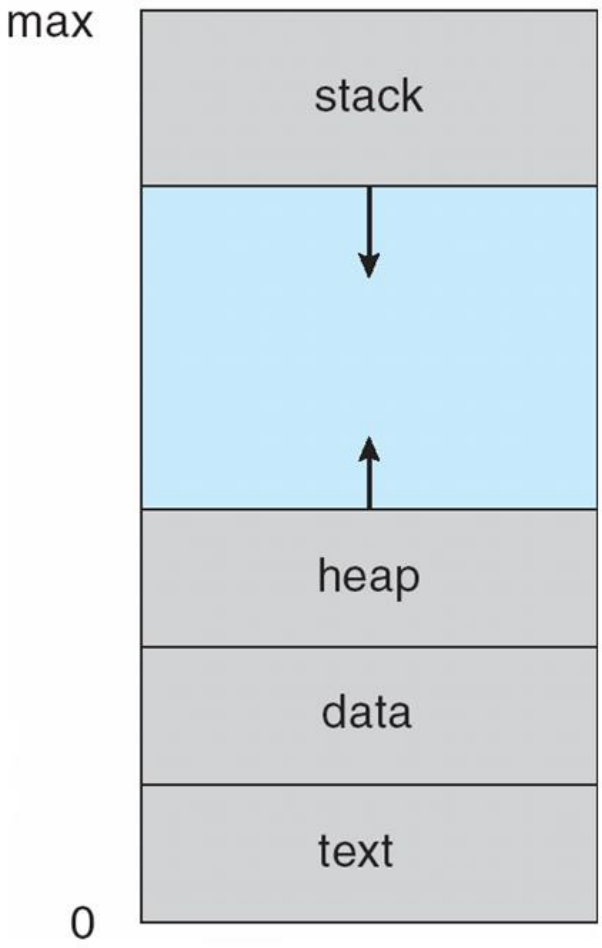
Process Concept (Cont.)

- Program is ***passive*** entity stored on disk (**executable file**), process is ***active***
  - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
  - Consider multiple users executing the same program





Process in Memory



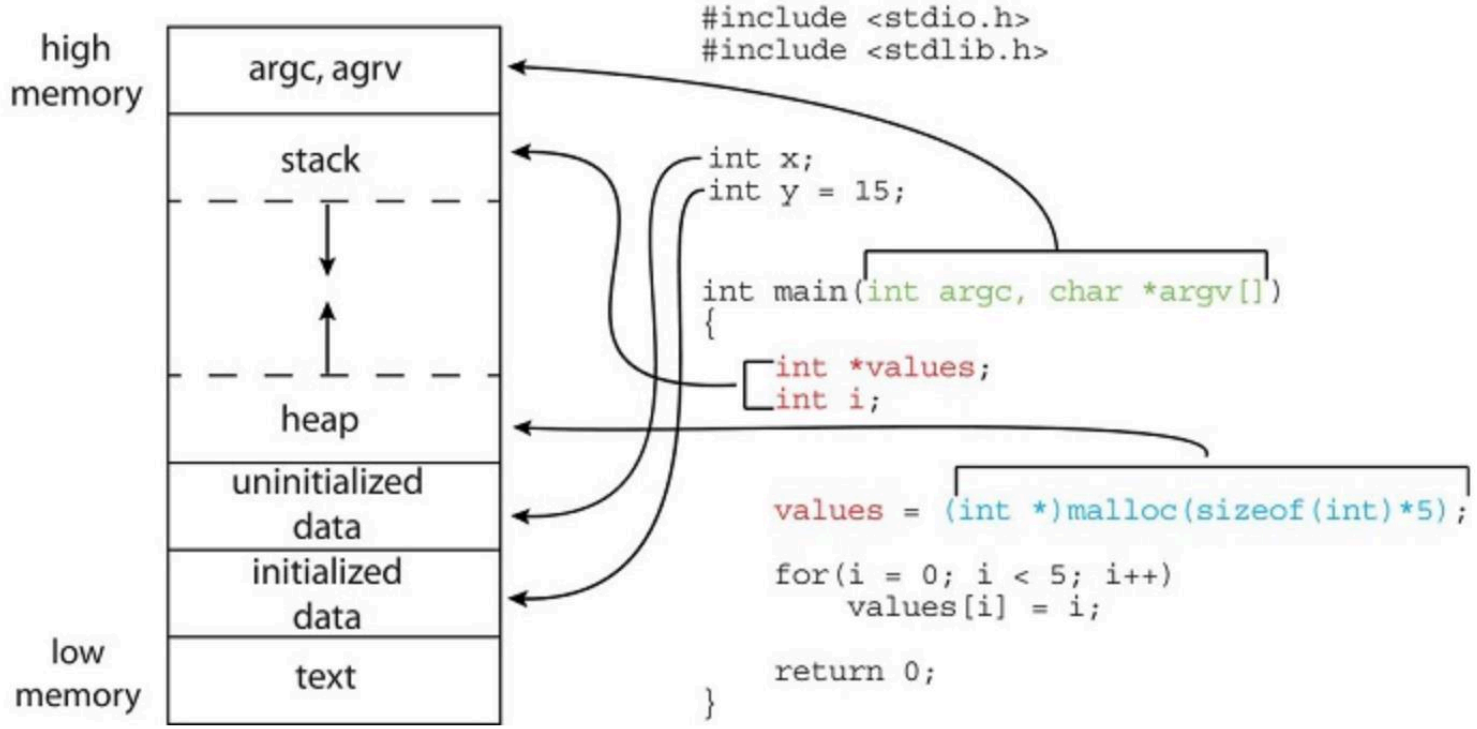
- ***Text section***
  - the executable code
- ***Data section***
  - global variables including static variables
- ***Heap section***
  - memory that is dynamically allocated during program run time
- ***Stack section***
  - temporary data storage when invoking functions (such as function parameters, return addresses, and local variables)





Memory Layout of a C Program

- The global data section is divided into different sections for (a) initialized data and (b) uninitialized data.
- A separate section is provided for the **`argc`** and **`argv`** parameters passed to the **`main()`** function.







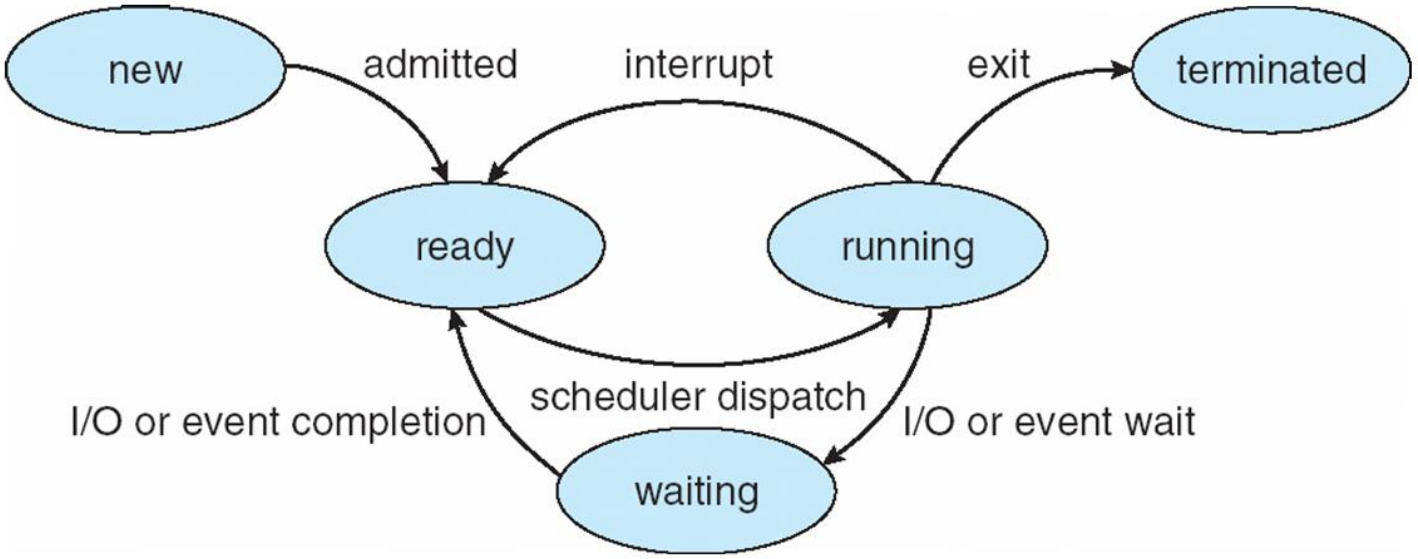
Process State

- As a process executes, it changes **state**
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution





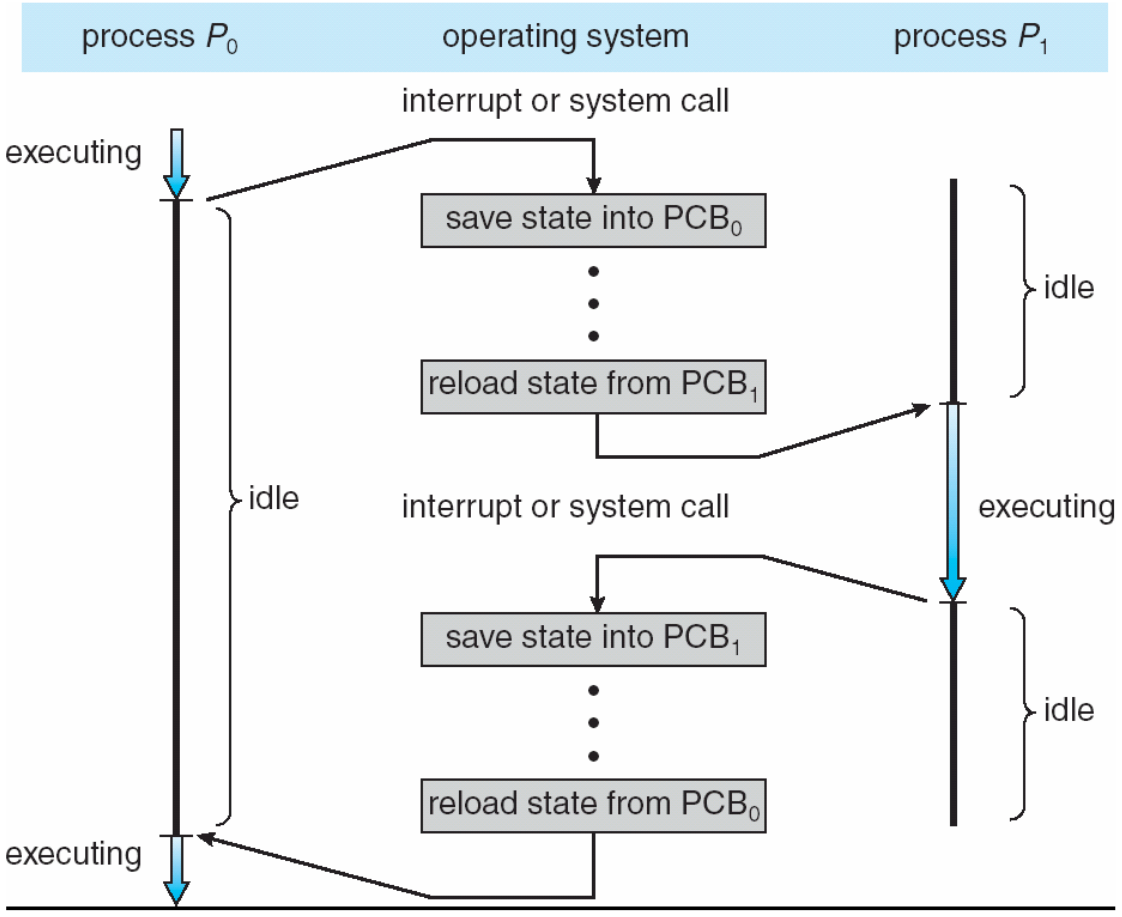
Diagram of Process State







CPU Switch From Process to Process







Example

```c
#include <stdio.h>

int main() {
    int a = 5;
    int b = 3;
    int sum = a + b;
    printf("P0: The sum is: %d\n", sum);
    int x = 10;
    int y = x * 2;
    printf("P0: Double of %d is %d\n", x, y);
    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int num;
    printf("P1: Please enter a number: ");
    scanf("%d", &num);
    int result = num * 10;
    printf("P1: You entered %d, multiplied by 10 is %d\n", num, result);
    return 0;
}
```





Threads

- So far, we talked about single-threaded process
- A thread is the smallest unit of execution that an operating system' CPU scheduler can run.
  - A thread lives inside a process.
- Each process has at least one thread
- Think of a **process** as a restaurant.
  - The **restaurant** is the process.
  - The **workers (chefs, waiters, cashiers)** are the threads.



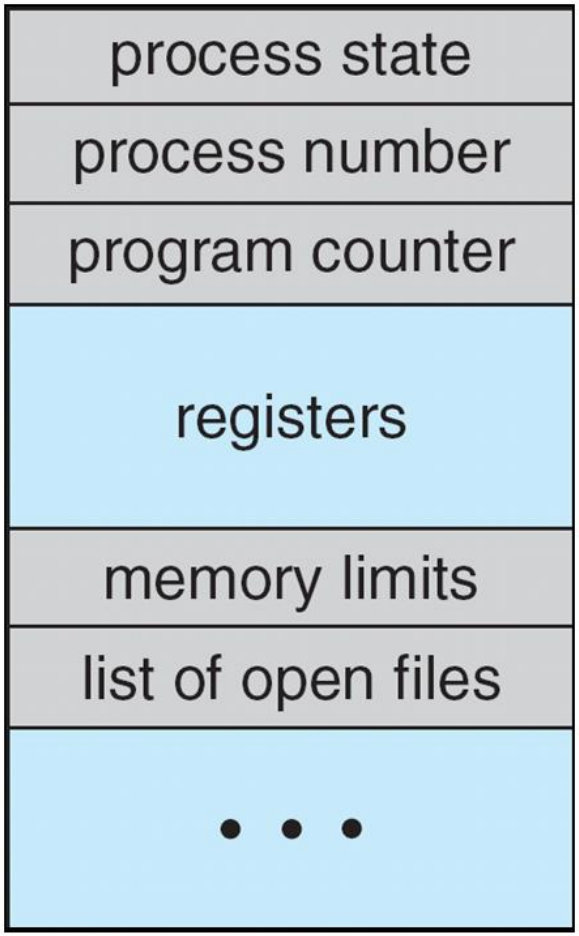


Process Control Block (PCB)

Information associated with each process

(also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files



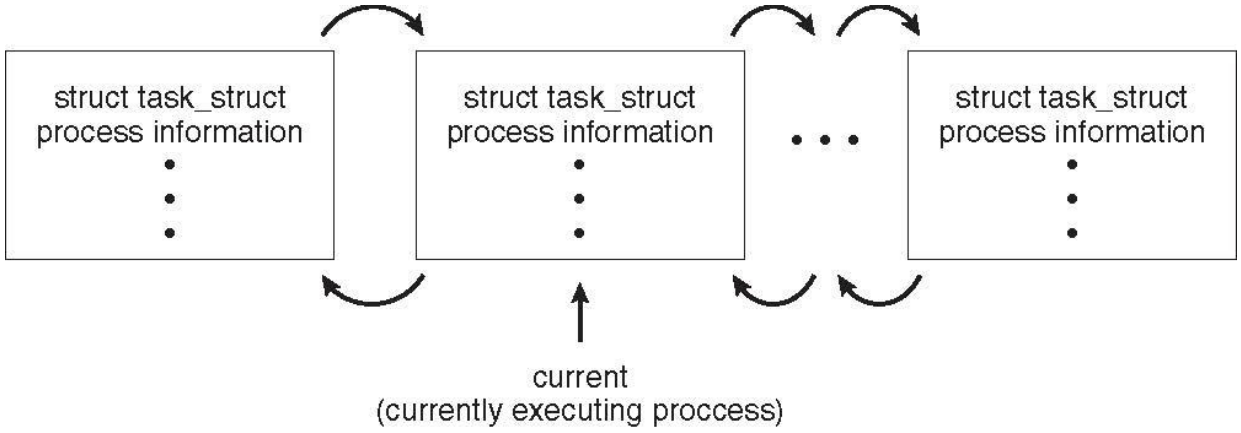




Process Representation in Linux

Represented by the C structure `task_struct`

```
pid t_pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```







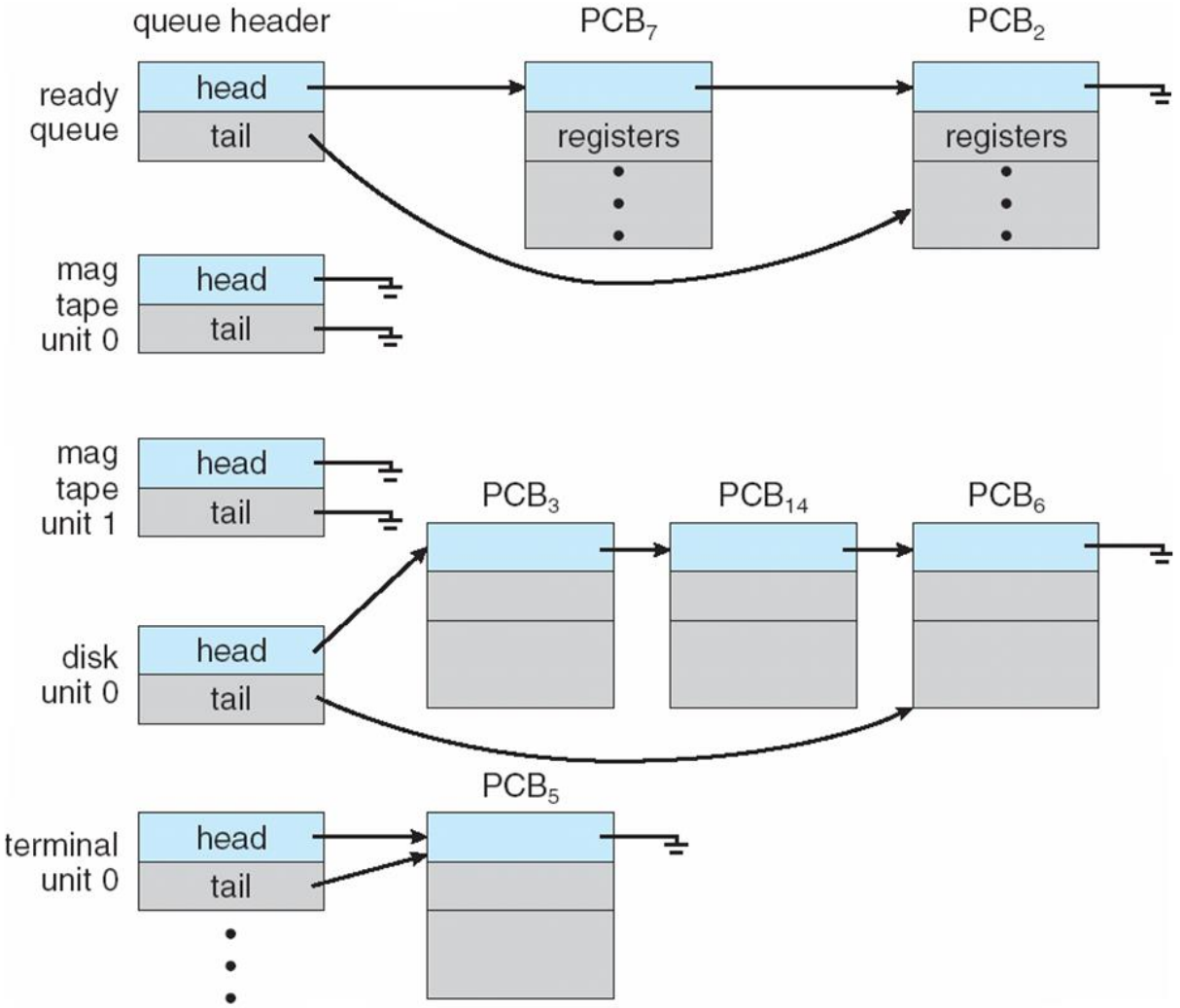
Process Scheduling

- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues





Ready Queue And Various I/O Device Queues

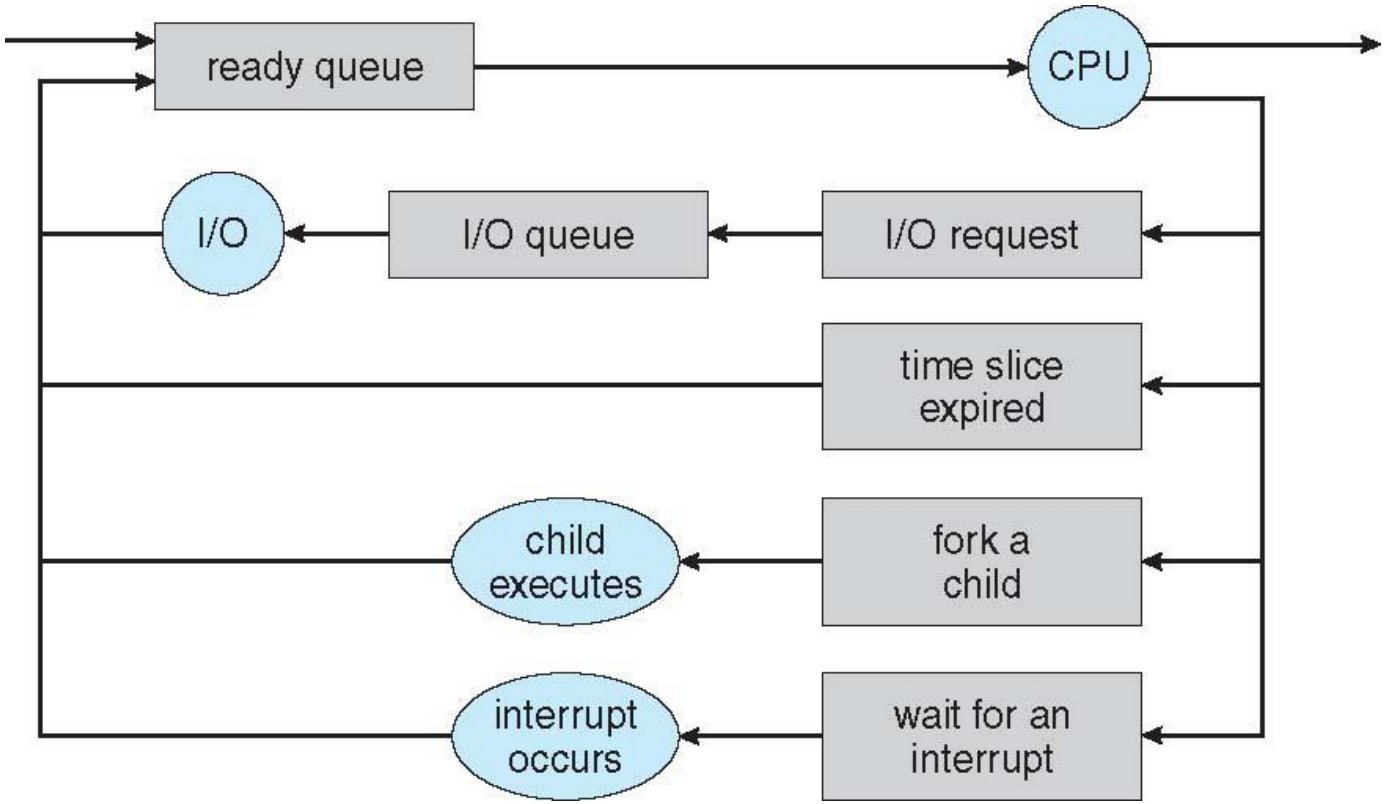






Representation of Process Scheduling

- **Queueing diagram** represents queues, resources, flows







Schedulers

- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - Sometimes the only scheduler in a system
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**





Schedulers

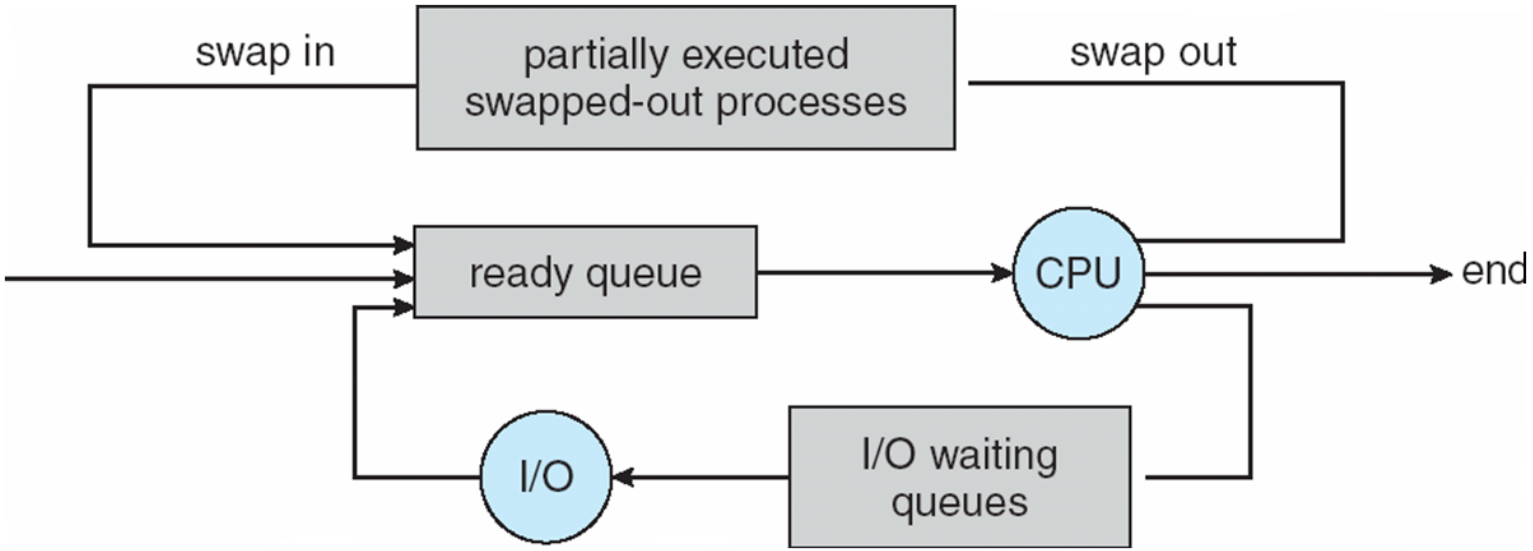
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***





Addition of Medium Term Scheduling

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
  - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**







Next …

- Review the last three slides on your own pace





Multitasking in Mobile Systems (1)

- Consider different ranks of processes:
  - **Foreground process** — user is interacting with it, or it is running a foreground service
  - **Visible process** — something the user can still see (paused activity, dialog over it, live wallpaper, IME). Almost as protected as foreground.
  - **Service process** — hosts a *started* service that is not a foreground service. Kept after foreground/visible processes, but no longer a safe long-term home.
  - **Background process** — activities are stopped; no running service that raises importance. First to be reclaimed (LRU).
  - **Empty process** — no components left; cached only for a faster restart.
- Same app can move between these ranks as the user does things.





Multitasking in Mobile Systems (2)

- Quick walk-through with one app (a music player):
  - You open it and tap Play → **foreground** (activity on screen).
  - You press Home; it promotes playback to a notification → still **foreground** (foreground service).
  - You stop the music and leave the app → **background**.
  - Android later destroys the leftover empty shell → **empty**, then gone.





Multitasking in Mobile Systems (3)

- Some mobile systems (early iOS) allowed only one process to run; the others were suspended or killed.
- Because of screen space and battery, iOS still treats the UI app as special:
  - **Foreground:** the app the user is interacting with (on iPhone usually one; on iPad, Stage Manager can show several).
  - **Background:** other apps stay in memory but are typically **suspended** (no CPU) unless they have an approved reason to run.
  - Allowed background work is narrow: short scheduled tasks (BGTaskScheduler), notifications, audio, location, navigation, and a few other declared modes.



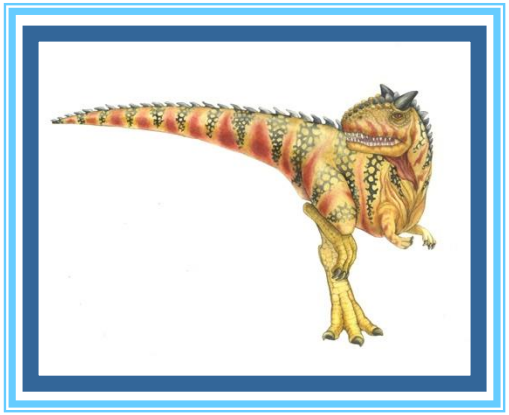


Multitasking in Mobile Systems (4)

- Android also has foreground vs background processes.
  - A **background process** is an easy target for the low-memory killer.
  - A Service is a component with **no UI**. It does **not** automatically keep running and does **not** get its own thread.
  - Long user-visible work (music, navigation, transfer) needs a **foreground service** plus a persistent notification.
  - Deferred or periodic work should use **WorkManager**, not a classic background Service.



Chapter 3: Process Part 2



The content of this session is from the textbook Chapter 3

Silberschatz, Galvin and Gagne ©2018



After completing this session, you should be able to:

- Given a C program using the POSIX API, construct a process tree that represents the program's execution.
- Given a C program using the POSIX API, calculate the total number of processes created during execution.
- Compare and contrast the main memory address space layouts of a parent process and its child processes.
- Determine whether a child process becomes a zombie or an orphan during program execution.
- Differentiate between the system calls fork() and exec() based on their behavior and effects on process execution.





Operations on Processes

- System must provide mechanisms for:
  - process creation,
  - process termination,
  - and so on as detailed next





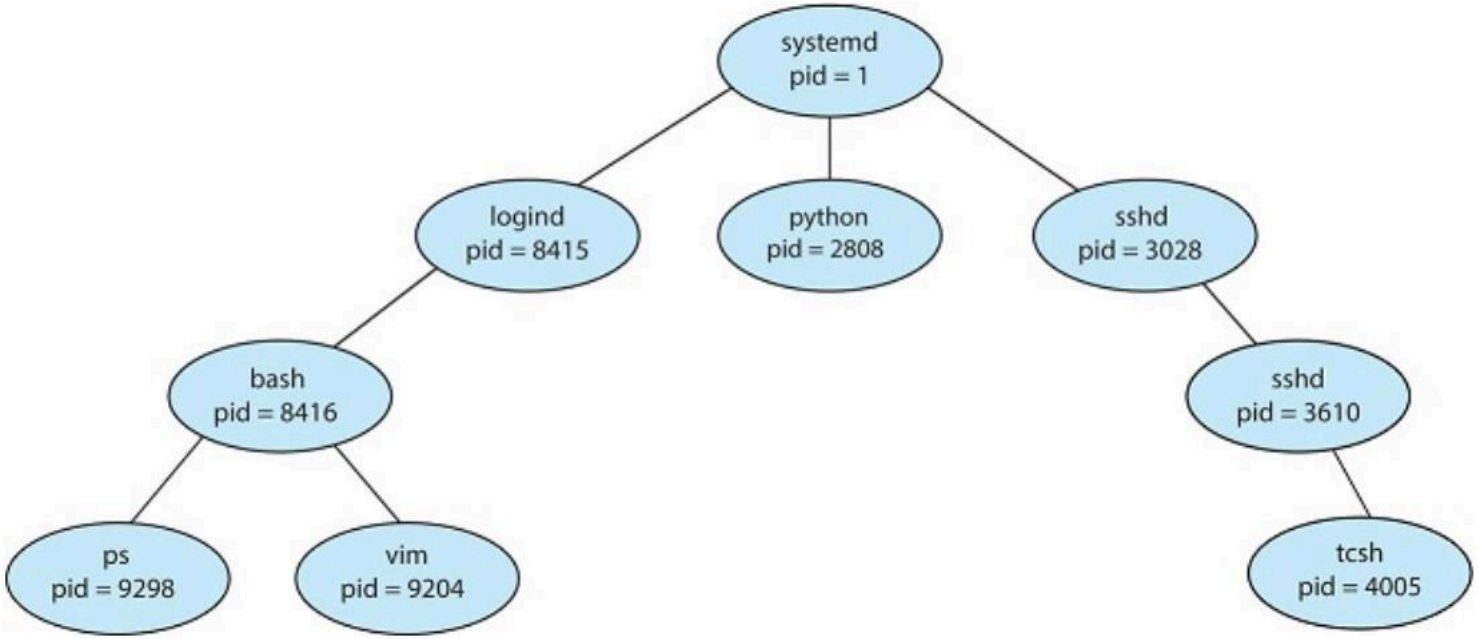
Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate





A Tree of Processes in Linux

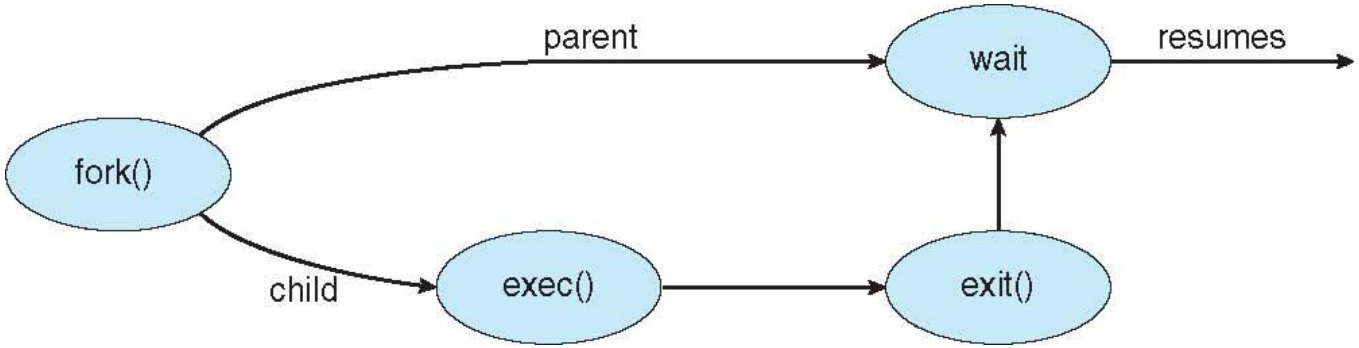






Process Creation (Cont.)

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - `fork()` system call creates new process
  - `exec()` system call used after a `fork()` to replace the process' memory space with a new program







C Program Forking Separate Process (Unix/Linux)

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```





Process Termination

- Process executes last statement and then asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates





Process Termination

- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc. are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the `wait()` system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

- If no parent waiting (did not invoke `wait()`) process is a **zombie**
- If parent terminated without invoking `wait`, process is an **orphan**





A Tree of Processes (Example 1)

process2.c

```c
#include<unistd.h>
#include<sys/types.h>
#include<stdio.h>

int main()
{
    pid_t p, q, r;
    printf("before fork 1\n");
    // create a child process
    p=fork();
    printf("before fork 2\n");
    // create a child process
    q=fork();
    printf("before fork 3\n");
    // create a child process
    r=fork();
    printf("after fork 1, 2, 3\n");
}
```





More on parent child processes - Example 2

```
1   int value = 5;
2   int main() {
3       pid t pid; pid = fork();
4       if (pid == 0) { /* child process */
5           value += 15;
6           return 0;
7       }
8       else if (pid > 0) { /* parent process */
9           wait(NULL);
10          printf("PARENT: value = %d", value); /* LINE A */
11          return 0;
12      }
13  }
```

What is the output of line 10 (LINE A)?





More on parent child processes - Example 3

```
1   int main(){
2       pid_t p;
3       p=fork();
4       if(p==0){
5           sleep(2);
6           printf("Child having id %d\n",getpid());
7       }
8       else
9       {
10          sleep(10); // Parent sleeps.
11          printf("Parent having id %d\n",getpid());
12      }
```

Does the child process in below program become orphan or zombie process? How can prevent it from becoming orphan or zombie?





More on parent child processes - Example 4

```
1   int main() {
2       pid_t pid, pid1,pid2;
3       /* fork a child process */
4       pid = fork();
5       if (pid == 0) { /* child process */
6           pid1 = getpid();
7           pid2 = getppid();
8           printf(pid); /* A */
9           printf(pid1); /* B */
10          printf(pid2); /* C */
11      }
12      else if (pid>0) { /* parent process */
13          wait(NULL);
14          pid1 = getpid();
15          printf(pid); /* D */
16          printf(pid1); /* E */
17      }
18      return 0;
19  }
```

Assuming that the actual pids of the parent and child are 2600 and 2603, respectively when the instance of this program run.

Identify the output of line 8 (A), line 9 (B), libe 10 (C), line 15(D), and line 16 (E)





Example 5

- Consider the following C code:

```
int main()
{
        printf("start of program");
        pid_t a, b;
        a =fork();
        b =fork();
        print("end of program\n")
}
```

- How many processes are created when the program is run?

